



Enhancing REST API fuzzing with access policy violation checks and injection attacks[☆]

Omur Sahin^{a,b}, Man Zhang^{c,d,*}, Andrea Arcuri^{b,d}

^a Erciyes University, Kayseri, Türkiye

^b Kristiania University of Applied Sciences, Oslo, Norway

^c Beihang University, Beijing, China

^d Oslo Metropolitan University (OsloMet), Oslo, Norway

ARTICLE INFO

Dataset link: <https://github.com/WebFuzzing/EvoMaster>, <https://github.com/WebFuzzing/Dataset>

Keywords:

REST
API
Fuzzing
Security
BOLA
BFLA

ABSTRACT

Due to their widespread use in industry, several techniques have been proposed in the literature to fuzz REST APIs. Existing fuzzers for REST APIs have been focusing on detecting crashes (e.g., 500 HTTP server error status code). However, security vulnerabilities can have major drastic consequences on existing cloud infrastructures.

In this paper, we propose a series of novel automated oracles aimed at detecting violations of access policies in REST APIs, as well as executing traditional attacks such as SQL Injection and XSS. These novel automated oracles can be integrated into existing fuzzers, in which, once the fuzzing session is completed, a “security testing” phase is executed to verify these oracles. When a security fault is detected, as output our technique is able to generate executable test cases in different formats, like Java, Kotlin, Python and JavaScript test suites.

Our novel techniques are integrated as an extension of EvoMASTER, a state-of-the-art open-source fuzzer for REST APIs. Experiments are carried out on 9 artificial examples, 8 vulnerable-by-design REST APIs with *black-box* testing, and 36 REST APIs from the WFD corpus with *white-box* testing, for a total of 52 distinct APIs. Results show that our novel oracles and their automated integration in a fuzzing process can lead to detecting security issues in several of these APIs.

1. Introduction

The Open Worldwide Application Security Project (OWASP) keeps track of the most common security vulnerabilities in web applications.¹ In their 2023-report targeted at web APIs,² the most common vulnerability is *Broken Object Level Authorization* (BOLA). A similar vulnerability, called *Broken Function Level Authorization* (BFLA), ranks fifth. Wrong authorization checks in the APIs can lead users to access resources they are not supposed to access (BOLA), or do operations they do not have rights to (BFLA).

Based on the “State of API Security 2026” made by 42crunch,³ which is based on 200 real-world API vulnerabilities and exploits discussed in the *APISecurity.io* newsletter coverage between 2024 and 2025, “the most frequent API vulnerability across all categories in 2025 was **Missing Authentication**. This occurs when an API fails to enforce

authentication for sensitive operations, allowing anonymous users to access data or perform actions that should be restricted to authenticated users”.³

As of 2024, it is estimated⁴ that 94% of companies worldwide use cloud computing, with costs measured in the hundreds of billions of euros. As such, reliability and correctness of these systems are of paramount importance in modern society (Addula et al., 2025).

Currently, REST APIs are the most common type of APIs on the web. As testing the functionalities of APIs can be expensive and time consuming, a lot of research has been carried out to automatically generate test cases for them (Golmohammadi et al., 2023). However, to deal with authorization-related faults, a fuzzer should be able to handle the authentication of different users with different roles and access rights. Unfortunately, many fuzzers in the literature are unable to handle authentication information during the fuzzing process (Sahin et al., 2025). For the few fuzzers that can handle authentication of

[☆] Editor: Tao Zhang.

* Corresponding author.

E-mail address: manzhang@buaa.edu.cn (M. Zhang).

¹ <https://owasp.org/www-project-top-ten/>

² <https://owasp.org/www-project-api-security/>

³ <https://42crunch.com/state-of-api-security-2026-report/>

⁴ <https://edgedelta.com/company/blog/how-many-companies-use-cloud-computing>

different users, they are unable to automatically detect BOLA and BFLA related vulnerabilities.

To fill this gap in the literature, in this paper we define seven novel automated oracles aimed at detecting faults related to misconfigured authorization checks. Furthermore, as the verification of authorization checks is only one aspect of security testing, we also integrated in our approach the evaluation of traditional SQL Injection and XSS attacks (which are hence not novel).

Many of these oracles are based on the correct use of the HTTP protocol. For example, non-authenticated calls should return a 401 HTTP status code, whereas an authenticated but not-authorized call should return a 403. Faulty APIs that use HTTP as a transfer protocol without conforming to its formal semantics would trigger our oracles without actually having security vulnerabilities. Still, those would be *critical functional faults* (e.g., returning a 200 status code on a failed call), which our techniques can detect and report to the user. Such misuses of HTTP may have nefarious consequences, such as reducing interoperability and misleading API clients. Proper *API Design* is a critical aspect for functional systems (Masse, 2011), where there is a lot of research work on automatically checking for API design and HTTP conformance (e.g., Di Meglio et al., 2025; Bogner et al., 2024; Decrop et al., 2026).

However, whether an automatically detected fault is a *security vulnerability* or a *critical misuse of HTTP* is something for the final user to decide, as it is based on the actual business logic and expected functionalities of the tested API. In either case, a serious fault is found that needs to be fixed.

We integrated our novel oracles in a state-of-the-art fuzzer for REST APIs, namely EvoMASTER (Arcuri et al., 2025c). Experiments on eight artificial APIs widely used in security research, as well as on 36 APIs from the WFD corpus (Sahin et al., 2025), show that our novel automated oracles can be used to automatically detect several new security-related faults that cannot be detected with other current approaches in the literature.

Our approach is integrated and evaluated in EvoMASTER, but any other state-of-the-art fuzzer that supports authentication of users (e.g., via authorization headers, or dynamic login tokens) could have been used for our study. We chose to use EvoMASTER as starting point for our implementation, as developing a robust fuzzer from scratch is a major engineering effort (Arcuri et al., 2023a). We chose EvoMASTER because it is among the oldest (Arcuri, 2017) and mature fuzzer still in active development (Arcuri et al., 2025c), giving among the best results in large tool comparisons (Kim et al., 2022; Zhang and Arcuri, 2023; Sartaj et al., 2024). As it is a mature tool with extensive documentation, it is used in several enterprises worldwide, such as Meituan in China (Zhang et al., 2023, 2024) and Volkswagen in Germany (Poth et al., 2025; Arcuri et al., 2025a). EvoMASTER has been downloaded thousands of times (Arcuri et al., 2025c).

This paper is an extension of the conference paper (Arcuri et al., 2025b). Compared with the conference version, we introduced six additional oracles (9 vs. 3) and conducted empirical studies on a larger set of APIs (52 vs. 13). In particular, in this extension we propose the novel oracles *Anonymous Modifications* (Section 3.4), *Ignore Anonymous* (Section 3.5), *Leaked Stack Traces* (Section 3.6) and *Hidden Accessible* (Section 3.7). We also integrated and empirically evaluated existing injection attacks such as *SQL Injection* (Section 4.1) and *Cross Site Scripting (XSS)* (Section 4.2). To properly evaluate all these oracles, we used a corpus that is four times as large as what previously used in Arcuri et al. (2025b). This makes this study the largest so far in the literature of security testing of REST APIs.

Besides the novel security oracles presented in this paper, EvoMASTER is also able to detect Server-Side Request Forgery (SSRF) vulnerabilities (Seran et al., 2026). However, the details of how it is done have been presented and analyzed in a different previous publication (Seran et al., 2026), and they are not going to be discussed here.

The paper is organized as follows. Section 2 discusses related work. Our novel security oracles are presented in Section 3. How we handle existing injection attacks is discussed in Section 4. How the use of these oracles can be integrated into a fuzzer is discussed in Section 5. Section 6 shows our empirical study. Threats to validity follow in Section 7. Finally, Section 8 concludes the paper.

2. Related work

There is a large body of research related to fuzzing REST APIs (Golmohammadi et al., 2023). Several fuzzers have been proposed in the literature, including, in alphabetic order: APIRL (Foley and Maffei, 2025), APIF (Wang and Xu, 2024), ARAT-RL (Kim et al., 2023), AutoRestTest (Kim et al., 2025b), bBOXRT (Laranjeiro et al., 2021), EvoMASTER (Arcuri et al., 2025c), LlamaRestTest (Kim et al., 2025a), MINER (Lyu et al., 2023), Morest (Liu et al., 2022), Nautilus (Deng et al., 2023), ResTest (Martin-Lopez et al., 2021), RestCT (Wu et al., 2022), RESTler (Atlidakis et al., 2019), RestTestGen (Viglianisi et al., 2020), Schemathesis (Hatfield-Dodds and Dygalo, 2022), VoAPI2 (Du et al., 2024) and WuppieFuzz (Rooijackers et al., 2025). These fuzzers aim at achieving high coverage (e.g., in terms of schema coverage and code coverage), and detecting faults. The most common type of oracle is detecting server errors (Marculescu et al., 2022), usually represented with a 500 HTTP status code (Golmohammadi et al., 2023). Other types of oracles include checking for schema mismatches (i.e., the test API is returning data that is not valid according to the API's schema), and robustness properties (i.e., sending wrong data on purpose, and then checking whether the API correctly flags them as user errors). In large empirical comparisons (Kim et al., 2022; Zhang and Arcuri, 2023), fuzzers for REST APIs are typically compared on code coverage and on the number of detected 500 status codes.

When it comes to verifying security properties in REST APIs, some initial work on this topic exists. One of the earlier approaches was in RESTler (Atlidakis et al., 2020), in which 4 security rules were defined. These included checking the semantics of HTTP operations (e.g., “A resource that has been deleted must no longer be accessible” (Atlidakis et al., 2020)) as well as checking user-namespace properties (e.g., “A resource created in a user namespace must not be accessible from another user namespace” (Atlidakis et al., 2020)). However, this latter rule might be too restrictive, and does not consider that there can be complex access control policies in place where some users might have access to some resources of other users (e.g., a typical case would be administrator users).

The work presented in Nautilus aims at detecting “vulnerabilities caused by improper user input handling” (Deng et al., 2023). However, the presented approach is not fully automated, as it requires the testers to manually annotate the OpenAPI schemas with extra information needed by Nautilus to detect vulnerabilities. VoAPI2 (Du et al., 2024) does different kinds of security attacks, including SSRF, XSS, and Command Injection testing. Different techniques are used to identify subsets of the API operations and parameters that are more likely to be affected by these kinds of security issues. Similarly, APIF (Wang and Xu, 2024) does different kinds of security attacks based on SecLists,⁵ for example, XSS, SQL Injection and SSRF. However, APIF is not able to handle OpenAPI schemas, and it requires first to capture live data via a reverse proxy.

“Mass assignment” is one of the most common security vulnerabilities in APIs, according to OWASP²⁰²³ report.² This issue happens when an attacker can update object properties that they should not have access to. The authors of RestTestGen (Viglianisi et al., 2020) provided an extension to it to enable detection of this kind of vulnerability.

The work in Pasca et al. (2025) presents an LLM-based approach to generate tests aimed at Broken Object Level Authorization (BOLA)

⁵ <https://github.com/danielmiessler/SecLists>

vulnerabilities in REST APIs. However, it is unclear how the automated oracles are designed. In particular, “*this does not represent the detection of actual BOLA vulnerabilities ... The potential for generating tests that incorrectly identify non-existent vulnerabilities needs further investigation*” (Pasca et al., 2025).

An alternative approach to detect BOLA vulnerabilities in REST APIs has been presented in Santos Filho et al. (2025). In this approach, OpenAPI schemas are transformed into Petri nets, and then server logs are used to analyze possible BOLA vulnerabilities. However, a limitation is that it “*requires having an OpenAPI specification with links. Unfortunately, this field is not widely used*” (Santos Filho et al., 2025).

Our work is significantly different from this existing literature. In our approach, we define a series of automated oracles aimed at detecting faults related to access control. The only requirements are the presence of an OpenAPI schema describing the API, and login information for at least two different users. Then, given a fuzzer that generates a set of test cases maximizing different criteria (e.g., schema and code coverage), we apply a post-processing strategy to generate new test cases designed to evaluate those automated oracles. With our approach, no further manual effort or custom schema annotation is required. As far as we know, none of these automated oracles presented in this paper have been proposed and evaluated in the literature so far.

Note that, in this work, we also discuss how we extended EvoMASTER to do SQLi and XSS attacks. Those are not novel, as they are based on existing malicious payloads from the literature,⁶ and they have been evaluated already in tools such as VoAPI2 (Du et al., 2024). However, a clear difference here is that in our approach we are able to generate *executable* test cases (e.g., in JUnit format) to enable testers/developers to verify and debug any found vulnerability.

There is a large body of research related to penetration testing, like for example the recent PentestGPT (Deng et al., 2024) and PentestAgent (Shen et al., 2025). However, typically such a line of work is focused on web applications, and it is unable to handle REST APIs. Furthermore, they focus on existing types of vulnerability attacks that are not related to access policy violations in REST APIs.

Regarding the status of BOLA in practice, Kaur provided a taxonomy based on more than 100 bug bounty disclosures (Kaur, 2026). Six different families of BOLA vulnerabilities are defined in the taxonomy. In many cases, it was possible to read or modify confidential data of other users by simply directly specifying resource IDs, which could be easily guessed or obtained via other endpoints.

3. Novel security oracles

In this paper, we define seven new automated oracles to detect different kinds of faults related to authorization and access control, as well as two further oracles related to executing SQLi and XSS attacks. To detect these faults, we need to generate test cases with specific scenario patterns, and then verify if the HTTP status codes and body responses we get back from the tested API might reveal the presence of these faults.

To enable some of these security oracles, authentication information for at least two different users is required by the fuzzer. Better if more (at least one per different role), especially considering different access policies they might have (e.g., administrators). No assumption on their relative access policies is needed, and no formal description of these policies is needed either. Such information is inferred dynamically based on the 401 (not authenticated) and 403 (not authorized) HTTP status codes received back during the fuzzing session. The functionalities of an API can be expressed with an OpenAPI⁷ schema, specifying which endpoints (combination of URL *paths* and HTTP *verbs*) are available, and what type of data they expect as input (e.g., query parameters

and JSON body payloads). However, OpenAPI schemas are not able to express information about access policies.

To simplify the explanations of how our novel automated oracles work, in our examples we will provide HTTP call sequences in the following format:

```
1 (AUTH) VERB PATH -> CODES
```

For example, a test case T_k could be:

```
1 (A) POST /items -> 201
2 (A) GET /items/42 -> 200
3 (B) GET /items/42 -> 403
```

In parentheses we have the user (where “?” means it does not matter), followed by the verb (e.g., GET) and the URL path, with then the resulting HTTP status code(s) of making this call. For simplicity, here we are omitting any query parameters, headers and body payloads. How those are handled will be discussed next. In this 3-step test case example T_k , a user A creates (status 201) a new item via a POST request, and then successfully accesses it with a GET request (status 200). However, another user B fails to access the same resource created by A (status 403) as they are not authorized.

To obtain calls on specific endpoints (e.g., POST /items) returning some specific HTTP status codes (e.g., 201), there might be a need for the right input data in the query parameters, body payloads and headers. If some data is not correct (e.g., according to some defined constraints), then the API most likely would return a 4xx user error status code. As such data and constraints could be arbitrarily complex, it is not necessarily the case that a fuzzer is able to generate such HTTP calls. When evaluating our novel automated oracles, we need to create test scenarios with specific HTTP call sequences, where each call might require to return some specific HTTP status code. To achieve this, we use and copy valid test cases generated during the fuzzing session. If for example a fuzzer is left running for 1-hour, and it generates n test cases T (with $|T| = n$) as output (based on maximizing some coverage criteria), we use those n test cases T_i as our initial pool to construct our new security tests.

Assume we need a call on GET /items/id that returns a 200 status code. This might not be trivial to obtain, if the call to generate such resource (e.g., via a POST or a PUT) requires complex input data. At the end of the fuzzing session, we can check if in the set T there is any test case with such an HTTP call. Assume the previous example T_k was generated. That would fit our needs. Nevertheless, all calls *after* our target call would not be necessary. Those unnecessary calls can be *sliced* away. However, calls *before* the target might or might not be needed, as they might modify the state of the API. Hence, it cannot be determined with certainty whether they are required. In this particular case, the first call POST /items creates the resource used in our target call GET /items/id. We can therefore make a copy C_k of T_k , and remove all calls after the target, but leave as they are all calls before. In this case, C_k would be:

```
1 (A) POST /items -> 201
2 (A) GET /items/42 -> 200
```

When generating security tests that identify the presence of faults, it is of paramount importance that users are properly informed about it. EvoMASTER can output test cases in different formats, including Java, Kotlin, JavaScript and Python. When a test case identifies a security fault, we make sure that in the generated test cases a comment is generated and added before the HTTP call triggering/showcasing the fault.

In the following subsections, we present in detail the seven novel oracles that are designed to detect the security faults. These include *not recognized authentication* (Section 3.1), *existence leakage* (Section 3.2), *missed authorization checks* (Section 3.3), *anonymous modifications* (Section 3.4), *ignored anonymous access* (Section 3.5), *leaked stack traces* (Section 3.6), and *hidden but accessible endpoints* (Section 3.7). To the

⁶ <https://github.com/minimaxir/big-list-of-naughty-strings>

⁷ <https://swagger.io/specification/>

best of our knowledge, no REST API fuzzer has proposed these oracles before (Arcuri et al., 2025b; Sahin et al., 2026).

In Section 4, we discuss how existing techniques for input-related vulnerabilities such as *SQL injection* (Section 4.1) and *cross-site scripting* (Section 4.2) can be integrated in our approach. Together, these oracles form a comprehensive detection for API vulnerabilities, covering multiple facets of API security, from authentication and authorization to information disclosure and input validation.

Moreover, to better explain these novel automated oracles, we will provide some examples of artificial APIs with seeded faults, on which we show what kinds of test cases can be automatically generated. Those are written in Kotlin, using the popular SpringBoot framework. All these examples are open-source and also used internally in EvoMASTER as E2E tests (and so part of its code repository), run as part of its Continuous Integration (Arcuri et al., 2023a), to verify EvoMASTER's own correctness (e.g., it should not crash, and its generated tests should be able to be compiled and run without issues).

Note that the code of the test cases that we will show are as generated by EvoMASTER version 5.1.0, as they are. No manual modification of any kind was done on any of these test cases for readability concerns in this paper.

3.1. Not recognized authentication

Typically, if an HTTP request is made without any authentication information, and the called endpoint requires authentication, then the API would answer with a 401 status code. If the user is authenticated, but, if they try to access a resource that does not belong to them, then the API should return a 403 status code (authenticated but not authorized).

If a user makes an authenticated call, but if then they receive a 401 status code (instead of for example a 403), then it is a fault. This is not a *security* vulnerability, but rather a *usability* fault. Based on the returned 401, the user might wrongly believe that their authentication info is wrong or outdated, and waste time in trying to resolve such an issue (e.g., by contacting the IT support of the API). As such, it would be best to fix this kind of usability fault.

In a fuzzer, simply flagging as a fault any 401 returned status code with an authenticated user would be unwise, as it could create many false-positives. For example, if a tester misconfigures the fuzzer, and provides a wrong user info, or such auth info has expired, then false-positive faults could be identified on each single API endpoint. Depending on the size of the API, this could result in hundreds of wrongly identified faults that could make it harder to highlight any found real fault.

To avoid this kind of false-positives, we designed the following strategy. First, for each endpoint X in the OpenAPI schema, we check if the fuzzer has generated any test case T_1 with an authenticated user (e.g., A) in which a 401 was returned. If so, X is potentially faulty. We then check if authentication information for A was correct. To do so, we check if the fuzzer has generated any test T_2 in which A was used on an endpoint $Y \neq X$ in which a 2xx, success status code was returned (i.e., no 401 for A). This is a first step to check the validity of A , as Y could be an open endpoint in which no authentication check is required. To verify this, we check if in the generated tests of the fuzzer there is any test T_3 in which any call on Y resulted in either a 401 or 403. If so, it would mean that Y requires authentication (T_3), the credentials of A on Y are correct as we get a 2xx status code (T_2), while the call on X with A identifies a fault as a wrong 401 is returned (T_1).

For each endpoint in the OpenAPI schema, if these three conditions are met, then we create a new test $K = \{C_3, C_2, C_1\}$, where C_i is a copy of test T_i , where all unnecessary calls after the target are sliced away, as previously explained.

Fig. 1 shows an example of generated test for an artificial API we developed to verify if our novel oracles can identify this kind of

faults. Two users are set up: FOO and BAR. According to the internal access policies of the API, the user FOO has no rights to call POST `/api/resources/`. However, instead of getting a 403 response, the last call in that test returns a wrong 401. The previous call shows that the authentication info for FOO was correct, i.e., this detected fault is not a false-positive due to a misconfiguration of the authentication information provided to the fuzzer.

3.2. Existence leakage

To clarify the issue with information leakage, let us consider an example. Assume that the fuzzing process generated at least one test case on an endpoint definition X in which a 403 status code is returned, and one test case in which a 404 is returned. The two test cases do not need to use the same or different users to get these results. For example, we could have X being something like `/data/id` where a GET call on `/data/42` returns a 403 status code, and a call on `/data/77` returns 404. These two calls could be done with the same user, or with two different users. Such a scenario is problematic, as it leads to information leakage on the existence of resources. If a user has no access to some resources (i.e., 403), they should not be able to learn which resources exist (403) and which ones do not exist (404), as such information can be used to narrow down follow up security attacks targeted at existing protected resources. The HTTP standards in RFC9110⁸ acknowledge the issue and explicitly state: “An origin server that wishes to “hide” the current existence of a forbidden target resource MAY instead respond with a status code of 404 (No. Found)”. For non-authorized users, to avoid information leakage either the APIs should always return 403 or always 404, regardless of whether the resource exists or not.

This is what is recommended to do by OWASP: “be careful that a malicious user is not able to glean any potentially sensitive information about the system from the status code returned”.⁹ We do not make an argument that either 403 or 404 should be returned, just that such decision should be consistent to avoid information leakage. For example, some authors suggest that: “it is preferred to return 404 not found => id does not exist, than 403 forbidden => id exists but not allowed, because such information should only be exposed to those who have access to the resource and the others are not supposed to know that it even exists”.¹⁰

However, there are cases in which a 404 does not lead to information leakage. This happens when the requested resource is a subresource that belongs to the user. Consider a new URL path Y , like for example `/data/id/bar`. Considering `/data/42/bar` returning 403, having `/data/77/bar` returning 404 with an authenticated user A does not necessarily mean a leakage. It could happen for example that a GET on `/data/77` returns 200, i.e., user A owns it and all its sub-resources like for example `/data/77/bar`. But what if `/data/77` does not exist (404)? Or the user A has no access to it (403)? Or A is actually no user (i.e., making a call with no authentication information, which should instead have led to a 401)? Then, we would have an information leakage.

To test this oracle, for each GET endpoint in the OpenAPI we need to create a new test representing the following scenario:

```
1 (?) GET /P/X -> 403
2 (A) GET /P/X -> 404
3 (A) GET /P -> 403, 404
```

Here we have two steps, with an optional third step. First, for each GET endpoint we check if we have at least one test case (T_1) returning 403 and one (T_2) returning 404. If yes, we can instantiate this scenario. We first make a copy of T_1 (including all input data, like query parameters and body payloads), and slice away all calls after that

⁸ <https://www.rfc-editor.org/rfc/rfc9110.html>

⁹ https://wiki.owasp.org/index.php?title=Exception_Handling&oldid=184270

¹⁰ <https://octo-woapi.github.io/cookbook/error-handling.html>

```

1 /**
2 * Calls:
3 * 1 - (201) PUT:/api/resources/{id}
4 * 2 - (403) PUT:/api/resources/{id}
5 * 3 - (201) PUT:/api/resources/{id}
6 * 4 - (401) POST:/api/resources/
7 * Found 1 potential fault of type-code 205
8 */
9 @Test @Timeout(60)
10 fun test_1_postOnResourcesAuthenticatedButWronglyToldNot() {
11
12     given().accept("/*/*")
13         .header("Authorization", "BAR") // BAR
14         .header("x-EMextraHeader123", "")
15         .put("${baseUrlOfSut}/api/resources/721")
16         .then()
17         .statusCode(201)
18         .assertThat()
19         .body(isEmptyOrNullString())
20
21     given().accept("/*/*")
22         .header("Authorization", "FOO") // FOO
23         .header("x-EMextraHeader123", "")
24         .put("${baseUrlOfSut}/api/resources/721")
25         .then()
26         .statusCode(403)
27         .assertThat()
28         .body(isEmptyOrNullString())
29
30     given().accept("/*/*")
31         .header("Authorization", "FOO") // FOO
32         .header("x-EMextraHeader123", "_EM_43_XYZ_")
33         .put("${baseUrlOfSut}/api/resources/577")
34         .then()
35         .statusCode(201)
36         .assertThat()
37         .body(isEmptyOrNullString())
38
39     // Fault205. Wrongly Not Recognized as Authenticated.
40     given().accept("/*/*")
41         .header("Authorization", "FOO") // FOO
42         .header("x-EMextraHeader123", "")
43         .post("${baseUrlOfSut}/api/resources/?EMextraParam123=_EM_45_XYZ_")
44         .then()
45         .statusCode(401)
46         .assertThat()
47         .body(isEmptyOrNullString())
48 }

```

Fig. 1. Example of generated test for an artificial API in which a not-recognized authentication fault is correctly identified.

target GET /P/X. A test case is composed of one or more HTTP calls. As previously discussed, calls after our target are not needed for our oracle. However, calls before the target might be required, as they might set the state of the API (e.g., by creating the resource). For example, T_1 could be something like:

```

1 (B) POST /data -> 201
2 (C) GET /data/42 -> 403
3 (B) GET /data/42 -> 200
4 (B) DELETE /data/42 -> 204

```

Then, after the slice, our copy C_1 would be:

```

1 (B) POST /data -> 201
2 (C) GET /data/42 -> 403

```

Then, we apply the same process to T_2 : we make a copy C_2 , where we slice all calls after the target. For example, assume we have T_2 being:

```

1 (A) GET /data/77 -> 404
2 (A) POST /data -> 201

```

Then, the copy C_2 would be just the first call. Then, we can create a new test K by combining C_1 and C_2 :

```

1 (B) POST /data -> 201
2 (C) GET /data/42 -> 403
3 (A) GET /data/77 -> 404

```

One possible issue here is that the execution of C_1 might have side effects on the execution of C_2 , as it might change the state of

the API (e.g., creating new data in the database, if any). This is not a problem when executing tests in isolation, as state-of-the-art tools such as EvoMASTER can automatically reset the state of databases after executing each test (Arcuri and Galeotti, 2020). This means that we must execute K , and verify that, for each call, we receive the same HTTP status codes as those returned in the original T_1 and T_2 tests. If the status codes remain the same, then we have detected a potential security-related fault.

However, we are not done, as we need to check if the 404 in the last call was valid, i.e., if the user owns any ancestor resource. If there is no authenticated user doing such call (i.e., in our example if A represents no user), then there is nothing more to check, and the fault is identified. Otherwise, given the called target endpoint, i.e., /data/id in this case, we check in the OpenAPI the top ancestor resource (in the URL path) that has a GET endpoint. If there is none, then there is nothing more to check, and the fault is identified. Otherwise, we create a new GET call on this ancestor resource using the same user authentication (e.g., A in this example). If we still get a 404, then the fault is identified. If not, the result remains inconclusive.

Fig. 2 shows an example of generated test for an artificial API we developed to verify if our novel oracles can identify this kind of faults. Two users are set up: FOO and BAR. When FOO accesses a resource that belongs to BAR, they get a 403 unauthorized response. However, accessing a non-existing resource on the same endpoint leads to a 404, which leads to information leakage.

```

1 /**
2 * Calls:
3 * 1 - (201) PUT:/api/resources/{id}
4 * 2 - (403) GET:/api/resources/{id}
5 * 3 - (404) GET:/api/resources/{id}
6 * Found 1 potential fault of type-code 204
7 */
8 @Test @Timeout(60)
9 fun test_5_getOnResourceAllowsUnauthorizedAccessToProtectedResource() {
10
11     given().accept("/*/*")
12         .header("Authorization", "BAR") // BAR
13         .header("x-EMextraHeader123", "")
14         .put("${baseUrlOfSut}/api/resources/12")
15         .then()
16         .statusCode(201)
17         .assertThat()
18         .body(isEmptyOrNullString())
19
20     given().accept("/*/*")
21         .header("Authorization", "FOO") // FOO
22         .header("x-EMextraHeader123", "")
23         .get("${baseUrlOfSut}/api/resources/12")
24         .then()
25         .statusCode(403)
26         .assertThat()
27         .body(isEmptyOrNullString())
28
29     // Fault204. Leakage Information Existence of Protected Resource.
30     given().accept("/*/*")
31         .header("Authorization", "BAR") // BAR
32         .header("x-EMextraHeader123", "")
33         .get("${baseUrlOfSut}/api/resources/339")
34         .then()
35         .statusCode(404)
36         .assertThat()
37         .body(isEmptyOrNullString())
38 }

```

Fig. 2. Example of generated test for an artificial API in which a leakage information fault is correctly identified.

3.3. Missed authorization checks

Resources should be protected from unauthorized access. For example, a user of a bank should not be able to do transactions on the accounts belonging to other users. However, access policies could be arbitrarily complex. Simply stating that a user should not access or manipulate resources of other users would just result in many false-positive faults.

The most obvious example is administrator users. A tester using a fuzzer could provide authentication information for some basic users as well as for an administrator one. This latter would be essential for testing endpoints that only administrators can have access to. An administrator would likely be able to access and manipulate resources of other users (albeit possibly still with some constraints). However, there are plenty of cases in which more complex scenarios would arise. For example, in a web application for discussion forums, the “moderator” for a specific forum might not have administration rights on the whole application. However, they might have special rights on all the messages created by other users in that forum (e.g., to delete them if they violate the terms of access of the application). Another example is websites related to children (e.g., government benefits or application for kindergarten), where groups of users (e.g., guardians for a child, typically their parents) share the same rights and resources, but cannot access the resources of other groups (i.e., other families). While these scenarios illustrate only a few examples, similarly complex access control rules can exist in various contexts.

Ideally, if there was a formal specification of these access policy rules, a fuzzer could use such specifications as an automated oracle, i.e., by checking if any rule is violated during the fuzzing session (e.g., getting a resource with a 2xx whereas it was supposed to get a 403). Unfortunately, we are aware of no existing formal specification for access policies in REST APIs which is widely used. Some languages

exist, like for example XACML¹¹ (used for example in Martin et al. (2006)), but we have never encountered any REST API using those formal specifications for access policies. Some languages/tools such as Open-Policy-Agent¹² and Cerbos¹³ seem to have some degree of popularity, but none of the open-source APIs we found, or industries we collaborate with, make use of them.

Without a formal specification telling us what the access policies are for a REST API, how can a fuzzer identify that a resource should have not been accessed? For example, if a user A accesses resource X and they get a 200 HTTP status code, how can a fuzzer tell that being a fault (i.e., a 403 was expected with no returned data)? This is an instance of the more general oracle problem: given $f(x) = y$, how can we be sure that y is the correct output for software f given the input x ? If f crashes, then clearly we have identified a fault for input x . But what about all other cases? The “oracle problem” is indeed one of the major challenges in software testing research (Barr et al., 2015).

In our particular case, we use the following strategy to address the oracle problem. In REST APIs, there are three distinct HTTP operators that can manipulate a specific resource: PUT (for whole updates), PATCH (for partial updates) and DELETE (for removing the resource). Our *hypothesis* is that, if any two of these operations are forbidden (i.e., 403), but the third is allowed (i.e., 2xx), then it is most likely a misconfiguration of the API. For example, an authorization check on an endpoint could have been forgotten. This would be a *major* security vulnerability, with possibly catastrophic consequences.

The intuition about this oracle is rather straightforward. Would it make any sense for a user to be forbidden to do any modification on a resource (e.g., with PUT and PATCH), but then be allowed to just delete it from the system with a DELETE? Likewise, would it make any sense

¹¹ <https://groups.oasis-open.org>

¹² <https://github.com/open-policy-agent/opa>

¹³ <https://github.com/cerbos/cerbos>

that a user is not allowed to DELETE a resource, but then be allowed to do any kind of changes on it that they want with a PUT or PATCH? Or could it be just that an authorization check is wrongly missing?

Among all possible APIs and different kinds of access policy rules, there could be cases in which these scenarios could happen and be correct. A possible example could be the case of a “Moderator” role that can DELETE content but cannot edit it using PUT requests. As such, we cannot exclude the presence of some false-positives when using these rules as automated oracles. Still, as they can be used to detect critical security faults, a non-zero amount of false-positive identified faults could be tolerable in practical contexts. As BOLA is the number one security vulnerability in APIs,² a certain amount of false positives could be acceptable. This is especially the case for APIs where access policies are not formally defined (e.g., using Open-Policy-Agent), and no other automated technique exists to point out such kind of issues. And, even in the case of false positives, that manual work needs to be done only once. Afterwards, in following fuzzing sessions, after a manual evaluation the oracle can be disabled for any given endpoint that leads to false positives.

Our approach works as follows. For each path X and each one of the verb V such as DELETE, PUT and PATCH, we check if it exists in the OpenAPI schema (not all defined paths might support all different HTTP verbs, such as for example PATCH). If so, we check in the generated tests T of the fuzzer if there is any T_k for which on that verb/path $V : X$ we get a 403 response. If T_k exists, we make a copy C_k in which all calls after the 403 are sliced away. If not, we try to create it. This is done by first checking for a test T_c that creates the resource at X , and then add a new call with a different user executing $V : X$ on it (due to its length and complexity, we do not provide here full details of all the edge cases dealing with such resource creation; we refer the interested reader to check our open-source implementation). If this latter call returns a 403, then such instantiated test will be our C_k . If not, our oracle is not applicable for $V : X$.

Given C_k in which the last call returns a 403 with a user A on path X for verb V , we need to add a new call $V' : X$ with same user A with a different verb V' out of the considered three. For example, if V is DELETE, for V' we consider PUT and PATCH. As making a success call with V' might require specific input data (e.g., query parameters and body payloads), we check if in T there is any existing T_j satisfying all these criteria. If not, we cannot apply our oracle on this path X . Otherwise, we make a copy C_j of T_j , where all calls *before* and after the target $V' : X$ are sliced away, i.e., C_j is composed of only a single call. Then, there are two further steps.

First, in C_j we modify the authentication information to rather use A (in case it was using a different user). Second, we need to make sure that both C_k and C_j handle the same resource. For example, C_k could deal with `/items/42` whereas C_j with `/items/77`. We create a new test Z which is the concatenation of C_k and C_j , where the resource path of C_j is *bound* to be the same as in C_k . For example, given C_k being:

```
1 (B) POST /items -> 201
2 (A) DELETE /items/42 -> 403
```

and C_j being:

```
1 (C) PUT /items/66 -> 200
```

then Z would be:

```
1 (B) POST /items -> 201
2 (A) DELETE /items/42 -> 403
3 (A) PUT /items/42 -> 200
```

The binding is done by either modifying the variables in the path elements of the endpoint URL, or by using the same dynamic updates used in the tests when resources are generated dynamically. For example, a resource could be created with a POST request, where the id of the newly generated resource could be dynamic and non-deterministic. The id could then be returned in the response of the POST request, e.g., in the location header or as a field in the body payload. Such

id needs to be extracted, and then used in the following HTTP calls for referring to the same resource. To achieve this, we simply rely on the existing algorithms in EvoMASTER to bind endpoints on same resources (Arcuri et al., 2025c).

Once Z is created, we execute it, and verify if the status codes are as expected, i.e., the DELETE returns a 403, followed by a PUT or PATCH on same resource with same user returning a 2xx. If this happens, then we have identified a potential security vulnerability.

Fig. 3 shows an example of generated test for an artificial API we developed to verify if our novel oracles can identify this kind of faults. Two users are set up: FOO and BAR. BAR creates a resource via a PUT. A different user FOO is forbidden from deleting this resource (403 on DELETE). However, this user can completely replace the resource with a PUT (as it gives a 204).

3.4. Anonymous modifications

The lack of authentication checks is one of the most common types of security vulnerabilities.³ However, it is not the case that all APIs, or all of their endpoints, must be secured. A read-only API (e.g., enabling just GET requests) might provide data without need of authentication credentials (e.g., an API for a web store, where items can be browsed before needing to login to buy them). Simply flagging as a vulnerability a GET endpoint, that returns a successful 2xx status code on a non-authenticated request, would just lead to false positives.

Also, it might be possible for APIs to create new data (e.g., via a POST call) without the need of authentication. A common example is when registering a new user in a system, or discussion forums that allow adding anonymous comments. Without any other available information, it is not really possible for a fuzzer to reliably decide whether GET/POST endpoints must require authentication or not.

However, this is not the case for modification operations, such as DELETE, PUT and PATCH. If anonymous (i.e., non-authenticated) DELETE operations are allowed, then a user could delete all existing data, and delete data of other users as soon as they create new entries. If anonymous PUT/PATCH operations are allowed, then as soon as a user creates some data, immediately another user could modify them, which would likely be detrimental for the original user.

Of course, to delete/modify the data of other users, somehow an attacker would need to determine the ids of those resources belonging to these other users. Depending on the complexity of those ids (e.g., incremental counters or UUIDs), these ids could be predicted, or directly retrieved if there are GET operations on collections of resources (e.g., to GET all the comments in a discussion forum).

Based on these intuitions, we can flag as potential security vulnerabilities any DELETE, PUT or PATCH endpoint for which we get a 2xx response without any valid authenticated user information. It might not be possible to be 100% sure that those are indeed vulnerabilities, i.e., for some special APIs those could be false positives. However, considering how common the issue of missing authentication is in industrial practice,³ those cases are definitively something worth to flag as potential issues to manually verify/review.

In contrast to the other oracles discussed previously, to flag this type of *Anonymous Modifications*, there is no need to create any special new test. We can simply check the tests generated during the fuzzing, and see, for each DELETE/PUT/PATCH endpoint, if any test was created in which a 2xx response was obtained without authentication credentials. An edge case is PUT returning a 201 (meaning resource is created). Besides doing modifications, by HTTP specs it is possible for a PUT to create a new resource. But, in such cases, the API is supposed to return a 201 status code. Therefore, we ignore those cases, i.e., anonymous PUT returning a 201 are not flagged as a security vulnerability.

Fig. 4 shows an example of generated test for an artificial API we developed to verify if our novel oracles can identify this kind of faults. A PUT was successful without any authentication info, and without creating any new resource (status code 204).

```

1 /**
2 * Calls:
3 * 1 - (201) PUT:/api/forbiddendelete/resources/{id}
4 * 2 - (403) DELETE:/api/forbiddendelete/resources/{id}
5 * 3 - (204) PUT:/api/forbiddendelete/resources/{id}
6 * Found 1 potential fault of type-code 206
7 */
8 @Test @Timeout(60)
9 fun test_7_putOnResourcMissedAuthorizationCheck() {
10
11     given().accept("/*/*")
12         .header("Authorization", "BAR") // BAR
13         .header("x-EMextraHeader123", "")
14         .put("${baseUrlOfSut}/api/forbiddendelete/resources/214")
15         .then()
16         .statusCode(201)
17         .assertThat()
18         .body(isEmptyOrNullString())
19
20     given().accept("/*/*")
21         .header("Authorization", "FOO") // FOO
22         .header("x-EMextraHeader123", "")
23         .delete("${baseUrlOfSut}/api/forbiddendelete/resources/214")
24         .then()
25         .statusCode(403)
26         .assertThat()
27         .body(isEmptyOrNullString())
28
29     // Fault206. Allowed To Modify Resource That Likely Should Had Been
30     // Protected.
31     val res_2: ValidatableResponse = given().accept("/*/*")
32         .header("Authorization", "FOO") // FOO
33         .header("x-EMextraHeader123", "")
34         .put("${baseUrlOfSut}/api/forbiddendelete/resources/214")
35         .then()
36         .statusCode(204)
37         .assertThat()
38         .body(isEmptyOrNullString())
39     val location_id_32 = res_2.extract().header("location")
40     assertTrue(isValidURIorEmpty(location_id_32));
41 }

```

Fig. 3. Example of generated test for an artificial API in which a missed authorization check fault is correctly identified.

```

1 /**
2 * Calls:
3 * (204) PUT:/api/resources/{id}
4 * Found 1 potential fault of type-code 901
5 */
6 @Test @Timeout(60)
7 fun test_12_putOnResourcAnonymousModifications() {
8
9     // Fault901. Anonymous Modifications.
10    given().accept("/*/*")
11        .header("x-EMextraHeader123", "")
12        .put("${baseUrlOfSut}/api/resources/196?EMextraParam123=42")
13        .then()
14        .statusCode(204)
15        .assertThat()
16        .body(isEmptyOrNullString())
17 }

```

Fig. 4. Example of generated test for an artificial API in which a PUT was successful without any authentication info.

3.5. Ignore anonymous

Automatically detecting *Missing Authentication* might not be possible with 100% certainty, as some APIs might have endpoints that do not require authentication (e.g., GET operation on data that is public, and not restricted to some specific users). The use of the *Anonymous Modifications* oracle (recall the previous Section 3.4) would cover some of the Missing Authentication cases, but not all.

A complementary approach is to check cases in which an API does verify users' credentials, but only if those are provided. For example, assume that on an endpoint we get a 401 or 403 when making an HTTP call with some credentials (where 401 would mean the credentials are invalid, e.g., like an expired JWT token). If it is possible to get a 2xx on the same endpoint without any credential, then that is a clear security vulnerability. This could happen if the API is wrongly configured to

verify the validity of credentials when provided, but then doing no authentication control if no credential is provided, i.e., the validity of anonymous requests is ignored.

To generate a test case to verify such a scenario for each endpoint, we do the following. First, we check if any test case T_1 was created with a 401 or 403 on such an endpoint for any given user A . If so, we make a copy C_1 , where all calls after the 401/403 are sliced away. Then, we check if there was any test case T_2 on the same endpoint in which a 2xx was returned with no authenticated user.

If T_2 exists, then we have found an *Ignore Anonymous* fault, which can be showcased with a new test with concatenated C_1 and C_2 , where C_2 is a sliced copy of T_2 .

If T_2 does not exist, we verify if any T_3 test case was generated in which the endpoint was covered with 2xx for a given different user B . If so, we make a sliced copy C_3 , where the credentials of B are removed.

```

1 /**
2 * Calls:
3 * 1 - (201) PUT:/api/resources/{id}
4 * 2 - (200) GET:/api/resources/{id}
5 * 3 - (403) GET:/api/resources/{id}
6 * 4 - (200) GET:/api/resources/{id}
7 * Found 1 potential fault of type-code 900
8 */
9 @Test @Timeout(60)
10 fun test_5_getOnResourceIgnoreAnonymous() {
11
12     given().accept("/*/*")
13         .header("Authorization", "FOO") // FOO
14         .header("x-EMextraHeader123", "42")
15         .put("${baseUrlOfSut}/api/resources/483")
16         .then()
17         .statusCode(201)
18         .assertThat()
19         .body(isEmptyOrNullString())
20
21     given().accept("/*/*")
22         .header("Authorization", "FOO") // FOO
23         .header("x-EMextraHeader123", "42")
24         .get("${baseUrlOfSut}/api/resources/483")
25         .then()
26         .statusCode(200)
27         .assertThat()
28         .contentType("text/plain")
29         .body(containsString("FOO"))
30
31     given().accept("/*/*")
32         .header("Authorization", "BAR") // BAR
33         .header("x-EMextraHeader123", "42")
34         .get("${baseUrlOfSut}/api/resources/483")
35         .then()
36         .statusCode(403)
37         .assertThat()
38         .body(isEmptyOrNullString())
39
40     // Fault900. A Protected Resource Is Accessible Without Providing Any
41     // Authentication.
42     given().accept("/*/*")
43         .header("x-EMextraHeader123", "42")
44         .get("${baseUrlOfSut}/api/resources/483")
45         .then()
46         .statusCode(200)
47         .assertThat()
48         .contentType("text/plain")
49         .body(containsString("FOO"))
50 }

```

Fig. 5. Example of generated test for an artificial API in which a protected resource is accessible without providing authentication.

If C_3 still leads to get a 2xx, the fault is found, which can be shown by concatenating C_1 and C_3 . For example, given T_1 as:

```

1 (?) POST /items -> 201
2 (A) DELETE /items/42 -> 403
3 ...

```

and T_3 as:

```

1 (B) POST /items -> 201
2 (B) DELETE /items/66 -> 200
3 ...

```

then, the resulting C_1 - C_3 would become:

```

1 (?) POST /items -> 201
2 (A) DELETE /items/42 -> 403
3 (B) POST /items -> 201
4 (-) DELETE /items/66 -> 200

```

Note that, simply removing the authentication credential A in T_1 to create C_3 (instead of searching for a different test T_3 that returned 2xx) might not work. This is because the call might require specific data (e.g., in query parameters and body payloads) to avoid 400 user error responses. So that is the reason why we try to start from a test T_3 that has no input validation errors (i.e., returning 2xx).

Fig. 5 shows an example of generated test for an artificial API we developed to verify if our novel oracles can identify this kind of faults. Two users are set up: FOO and BAR. FOO creates a resource via a PUT,

and it is able to successfully retrieve it with a GET. A different user BAR is forbidden from accessing this resource (403 on GET). However, trying to access such protected resource without any authentication info is successful (i.e., 200 on GET).

3.6. Leaked stack traces

Software faults are common. In a REST API, when an application crashes due to a fault (e.g., a null-pointer exception), then typically only the application layer of the API fails, and a 500 HTTP status code is sent back to the client (i.e., the entire HTTP server does not crash, so the TCP connection is still open and an error message can be sent back).

To help debug the fault, the server could log the *stack trace* of the exception to a log file, and/or send it back in the body payload of the HTTP 500 response. Where this latter behavior can be useful during the development of an API (e.g., while an engineer is developing a new feature, like a new endpoint), it can be a major security vulnerability for APIs run in production. An attacker that triggers a 500 fault (e.g., by using a fuzzer) would be able to see such stack traces. From the stack traces they could infer the programming language and framework used to develop the API. Such information could then be used for more targeted attacks (e.g., if that specific framework has known zero-day vulnerabilities).

```

1 /**
2 * Calls:
3 * (500) GET:/api/resources/null-pointer-json
4 * Found 2 potential faults. Type-codes: 100, 902
5 */
6 @Test @Timeout(60)
7 fun test_1_getOnNull_pointer_jsonShowsFaults_100_902() {
8
9     // Fault100. HTTP Status 500. com/foo/rest/examples/spring/openapi/v3
10    // /security/stacktrace/json/
11    StackTraceJSONApplication_34_nullPointerException
12    // Fault902. Leaked Stack Trace.
13    given().accept("application/json")
14        .header("x-EMextraHeader123", "")
15        .get("${baseUrlOfSut}/api/resources/null-pointer-json?
16        EMextraParam123=_EM_0_XYZ_")
17        .then()
18        .statusCode(500) // com/foo/rest/examples/spring/openapi/v3/
19        security/stacktrace/json/StackTraceJSONApplication_34_nullPointerException
20        .assertThat()
21        .contentType("application/json")
22        .body("'error'.type", containsString("java.lang.
23        NullPointerException"))
24        .body("'error'.message", containsString(""))
25        .body("'error'.stack.size()", equalTo(56))
26        .body("'error'.stack", hasItems("java.lang.
27        NullPointerException", "\tat com.foo.rest.examples.spring.openapi.v3.
28        security.stacktrace.json.StackTraceJSONApplication.nullPointerException(
29        StackTraceJSONApplication.kt:36)", "\tat jdk.internal.reflect.
30        GeneratedMethodAccessor90.invoke(Unknown Source)", "\tat java.base/
31        jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(
32        DelegatingMethodAccessorImpl.java:43)", "\tat java.base/java.lang.
33        reflect.Method.invoke(Method.java:569)", "\tat org.springframework.
34        web.method.support.InvocableHandlerMethod.doInvoke(
35        InvocableHandlerMethod.java:197)", "\tat org.springframework.web.
36        method.support.InvocableHandlerMethod.invokeForRequest(
37        InvocableHandlerMethod.java:141)",
38        ...

```

Fig. 6. Excerpt of a generated test for an artificial API in which an exception stack-trace is present in the response body. Due to long stack-trace (56 lines), the test is not displayed in its entirety.

As a rule of thumb, HTTP 500 status responses should provide no information to the clients about any internal details of the API, as such information can be exploited by malicious actors. Developers could enable stack traces in the responses during development, and, then, forget to disable such a feature when the API is deployed to production. This is a security vulnerability.

To verify whether an HTTP 500 response contains stack traces, we developed a series of regular expressions targeted at all the major programming languages (e.g., Java, Python, C#, JavaScript, Go, PHP, Ruby and Rust). Stack traces could be returned in plain-text, or embedded in a JSON response. Both cases are handled.

As such text analyses have a non-negligible computational cost, they are not executed on each HTTP 500 response. During a 1-24 h fuzzing campaign, hundreds of thousands of such tests could be evaluated. Once the fuzzing session is completed, for each endpoint we check if any test was generated returning a 500. If so, the text analysis is applied only on that test for that endpoint. Given N different endpoints in the API, these text analyses are run at most N times on N different tests.

When doing fuzzing with EvoMASTER, by default the *Leaked Stack Traces* is activated. However, in case of its use by developers that willingly want to have stack traces in their HTTP responses, such oracle should be deactivated, as it would otherwise generate only false positives. EvoMASTER is a highly tunable fuzzer, with hundreds of available (and documented¹⁴) options (Arcuri et al., 2023a). The use of each single oracle can be toggled via configuration settings (command-line input arguments and/or configuration files).

Fig. 6 shows an example of a test case that leads to an internal crash (HTTP status code 500), in which the stack-trace ends up in the response body. Here, two distinct faults are found and flagged: the fact

that the API crashed (status code 500), and that a stack-trace leaked in the response.

3.7. Hidden accessible

An OpenAPI schema defines what the API can provide. What if the API has working endpoints that are not declared in the schema? This could be a bug in the schema itself, i.e., the designers/developers of the API forgot to specify the endpoint. This can be the case when schemas are written manually (and not automatically derived from the source code of the API, e.g., using libraries such as SpringDoc for SpringBoot applications). Although this is not a security issue, it is still a *usability* fault that requires fixing.

However, it might also be that those endpoints are not supposed to be accessible. They could be used just for debugging/testing, or being work-in-progress. Having those endpoints accessible in production by users could become a security risk. If an endpoint is “hidden” (i.e., not declared in the schema), but it is still “accessible” when clients call it directly, we have a problem (either a usability or a security one).

If a client makes a call to an endpoint for a resource that exists, but the “verb” used is not supported for it, then the server should return either an HTTP 405 status code (*Method Not Allowed*) or a 501 (*Not Implemented*). To limit disclosing information for security reasons, the server might even decide to return a 403 in those cases instead. If a call is made to an endpoint that is not declared in the schema, but the response contains a status different from 403, 405 and 501, then there is a problem.

To check these properties, we use the following approach. For each distinct resource path declared in the schema, we do make a OPTIONS HTTP call. This will provide information about the available verbs for that path, returned in the Allow HTTP header. We then check if there is any allowed verb not declared in the schema (excluding OPTIONS itself and HEAD). If so, for each of those hidden verbs, we

¹⁴ <https://github.com/WebFuzzing/EvoMaster/blob/master/docs/options.md>

```

1 /**
2  * Calls:
3  * 1 - (200) OPTIONS:/api/resources
4  * 2 - (200) GET:/api/resources
5  * Found 3 potential faults. Type-codes: 101, 903
6  */
7 @Test @Timeout(60)
8 fun test_1_getOnResourcesShowsFaults_101_903() {
9
10     // Fault101. Received A Response From API With A Structure/Data That
11     // Is Not Matching Its Schema. Type: validation.request.operation.
12     // notAllowed OPTIONS operation not allowed on path '/api/resources'.
13     given().accept("/*/*")
14         .options("${baseUrlOfSut}/api/resources")
15         .then()
16         .statusCode(200)
17         .assertThat()
18         // .header("Allow", "HEAD,POST,GET,OPTIONS")
19         .body(isEmptyOrNullString())
20
21     // Fault101. Received A Response From API With A Structure/Data That
22     // Is Not Matching Its Schema. Type: validation.request.operation.
23     // notAllowed GET operation not allowed on path '/api/resources'.
24     // Fault903. Hidden Accessible Endpoint.
25     given().accept("/*/*")
26         .get("${baseUrlOfSut}/api/resources")
27         .then()
28         .statusCode(200)
29         .assertThat()
30         .contentType("text/plain")
31         .body(containsString("OK"))
32 }

```

Fig. 7. Example of generated test for an artificial API in which first a call is made via OPTIONS to determine which methods are allowed, followed by a successful call to a GET that is not defined in the schema.

make a call with it on that resource path, with no body payloads and no query parameters (as there is no info about them in the schema). If we obtain a result different than a 403, 405 or 501, we flag it as an error. To reproduce this problem, we instantiate a new test case with the OPTIONS call, followed by the call with the hidden verb.

Technically, the call with OPTIONS might be redundant. One could simply directly call all endpoints with verbs not declared in the schema, and see what is returned as status code. The use of OPTIONS just makes the tests more clear, better highlighting the severity of the issue.

Fig. 7 shows an example of test in which first an OPTIONS call is made on a path resource. Based on the Allow header in the response, considering that no GET is declared in the schema (not shown), a follow up call with a GET is done. As this is successful, such call is marked as a potential security vulnerability (besides being as well a violated schema fault).

4. Existing input validation oracles

There are different types of existing security testing attacks, such as SQL Injection and XSS. Those are not novel. However, in the context of REST APIs, it is not necessarily trivial to create successful attacks of this type. REST APIs might still require to generate correct input data that pass their first layer of input validation (e.g., strings that are supposed to represent dates, and numeric values constrained in specific ranges). Especially when those constraints are not specified in the APIs' schemas, you need fuzzing and different kinds of advanced techniques to generate such kind of data. As such, although this type security attacks (i.e., SQLi and XSS) are not novel, how they can be successfully integrated as part of a fuzzing process requires investigation.

4.1. SQL injection

In 2025, *Injection* is still one of the most common security vulnerability in web applications,¹ although its presence in APIs is less prevalent.² In the research literature, there is a large body of knowledge regarding how to automatically detect injection vulnerabilities such as SQL Injection (e.g., Kieyzun et al., 2009; Appelt et al., 2014; Aziz et al., 2016; Liu et al., 2020), including work on REST APIs (Du et al., 2024).

In our work, we do not only try to detect security vulnerabilities, but we also aim at being able to generate executable test cases to enable engineers to effectively debug such faults. In our approach, for each endpoint, we check if any test case T_i was generated in which a successful 2xx response was obtained. Responses that end up with user errors (i.e., 4xx) or server errors (5xx) might be less likely to involve database interactions. Then, for each input variable that is a string (e.g., in query parameters and fields in body payloads), we append a malicious SQL injection payload. This is as long as such extended string is still valid for any constraints expressed in the OpenAPI schema (e.g., length, enum and pattern constraints, if any are defined), as otherwise the API will just return a 400 status code without executing any business logic.

We need to make sure that we can automatically detect if a SQL Injection attack is successful, also when dealing with black-box testing where no direct access to the database is possible. For this case, we focus on *sleep* payloads, like for example ' OR SLEEP(5) -- '. Given N endpoints and P different malicious payloads to evaluate (with $P = 16$ in our case), at most $N \times P$ new test cases need to be evaluated. In the case of white-box testing, in which we can detect if the API is using any SQL database, if no database is used, then the evaluation of this *SQL Injection* oracle is automatically skipped.

If the injection of a sleep of S seconds is successful, then the test case should take at least S seconds longer to run. However, execution time of a test case for a REST API is non-deterministic, as it depends on many factors such as network load, and OS CPU schedule. Even when re-executing the same test twice, a certain amount of time variability is expected. The injected sleep time should be long enough to minimize risks of this source of variability, but not too long to make the testing unpractical. Ideally, HTTP calls to an API should just take few tens or hundreds of milliseconds. In those cases, a sleep of few seconds would unlikely to be affected by noise. But, if for any reason the execution time of HTTP calls for some endpoints are expected to be taking seconds, then such time variability might have negative effects. To handle this, we exclude from our analysis any HTTP call that is taking longer than M seconds (e.g., $M = 2$). Furthermore, EvoMASTER has a built-in mechanism to handle flakiness in its generated tests (Zhang

et al., 2026). If for any reason an HTTP call that would take just few milliseconds all of a sudden takes more than S seconds due to some OS noise, it would be caught and excluded when re-executed in EvoMASTER's flakiness checks.

To create test cases to find and showcase SQL Injection vulnerabilities, we do as follows. For each endpoint, we check if any test case T_1 was created where an HTTP call returned a 2xx and lasted less than M seconds. If any is found, we make a copy C_1 in which any call after the target (let us call it X) is sliced away. Then, we make a copy of the target HTTP call (let us call this copy Y), in which we inject malicious payloads, and append it to C_1 . If the target HTTP call X required the ids of dynamically generated resources in the test, we need to make sure the appended copied call Y refers to the same dynamic ids. For example, given T_1 for the endpoint GET /items/{id}:

```
1 (?) POST /items -> 201
2 (?) GET /items/42 -> 200
3 ...
```

then the resulting C_1 would be:

```
1 (?) POST /items -> 201
2 (?) GET /items/42 -> 200
3 (?) GET /items/42 -> 200
```

When evaluating C_1 , if X takes less than $M = 2$ seconds, and Y takes more than $S = 5$ seconds, then we have detected and can reproduce the SQL Injection vulnerability. Note, to be able to properly convey this information to the final users (e.g., developers and testers), our output test suites are enhanced with time tracking of the HTTP execution times, with assertions based on the values M and S .

Fig. 8 shows an example of a test case with two HTTP calls. The first call is executed, and we make sure to verify it takes less than 2 s (Line 8). The second call is a copy of the first, but with some SQL Injection payloads. Here, we verify the call takes more than 5 s (Line 57).

Even when the time-checks are long, there is no formal guarantee that no network latency or database load variability could negatively impact this oracle. Still, the presence of a false positive would be extremely unlikely. For that to happen, the following sequence of events should unfold: (1) during fuzzing, the first call with no SQLi payload should have no jitter (so the 2-second timeout is not reached), whereas the second call does have a random jitter (i.e., 5-second timeout is reached); (2) at the end of the search, when test cases are re-executed one or more times for flakiness checks (Zhang et al., 2026), the same case of (1) should happen again, i.e., random jitter for the second HTTP call but not the first; (3) when the fuzzing is completed, and the test engineers run the generated test for debugging the SQL Injection fault, again there should be no jitter for the first call (otherwise the assertion on Line 32 in Fig. 8 would fail), and a jitter when executing the second call (otherwise the assertion on Line 57 would fail). On the one hand, random jitters on networks and databases are possible. On the other hand, a series of random jitters that happen only at precise points in time, at microsecond precision, are less of a practical concern. In none of our experiments we saw any false positive regarding our SQL Injection checks, as those occurrences would be extremely rare.

4.2. Cross Site Scripting (XSS)

Cross-Site Scripting (XSS) is a widespread type of security vulnerability where attackers could inject malicious client-side scripts (usually JavaScript) into the frontend of web applications. As it is a common vulnerability in web applications, a significant body of research has been dedicated to analyzing and designing techniques to identify it and prevent it (Hydara et al., 2015). For example, a payload like `<img src=x onerror=alert('XSS')>` would execute an alert command if displayed directly in the HTML of a web page without escaping first the opening/closing tag characters.

In the case of REST APIs, there is no web frontend, though. As such, the API itself is not subject to XSS vulnerabilities. However, the

API “might” store malicious XSS payloads that, if later retrieved by a faulty web frontend application, they “might” result in XSS exploits if not escaped. As such, *Stored XSS* payloads are a potential threat, if malicious payloads are saved as they are without any input sanitization. This could happen with operations like POST, PUT and PATCH.

Besides storing malicious XSS payloads, *Reflected XSS* are problematic as well. A GET request with a malicious payload would be dangerous if the payload is returned as part of the response, and if the response is displayed without sanitization. A typical example of this problem is in search web forms where the searched query is displayed with its results. Triggering a malicious search could be as easy as clicking on a compromised link, if the searched query is passed by URL query parameters.

To create test cases to flag potential XSS vulnerabilities, we do as follows, using a similar approach as done for *SQL Injection* handling. For each endpoint, we check if any test case T_1 was created returning a 2xx. We make a copy C_1 where we slice away any call after the target endpoint. Given P different XSS payloads, we apply them to each string input, i.e., we replace any string value (e.g., in query parameters and body payloads), but only as long as the OpenAPI constraints are satisfied (e.g., on string length, enumeration and pattern constraints). If the execution of this new test C_1 still returns an HTTP 2xx, and the malicious payload is present in the body payload of this response as it is (i.e., with no sanitization), then we have detected a potential *Reflected XSS*.

For the case of *Stored XSS*, this approach might not be enough, as the results of POST, PUT and PATCH might, or might not, return any data in the body payload of their responses (e.g., a POST could return a representation of the created resource, including a dynamically generated id on the server-side, but that is not a requirement). Furthermore, even if data is returned, it does not mean that such data was actually *stored* in the backend system of the API. To handle these cases, we need to append to C_1 a GET request on the same resource manipulated/created by the POST, PUT and PATCH. In this GET call, we apply the same P payloads, and verify if its execution lead to see back the any of this payload as they are, without any input sanitization.

Given N endpoints, at most $N \times P$ new test cases are generated and evaluated to verify the presence of potential XSS vulnerabilities.

Fig. 9 shows an example of generated test case in which a resource is created with a POST, where some XSS payloads are passed as query parameters. This call is followed by a GET on the same resource, in which we verify that the XSS payload appears in the response's body.

5. Fuzzer integration

To use our novel oracles, we need to create specific scenarios, and then verify if any fault is detected based on the HTTP status codes and body responses we receive back from the tested API. To check and verify authentication and authorization properties, we need to be able to generate test cases that can exercise all the different functionalities of the API. Input data can have complex constraints (e.g., in query parameters and body payloads), and so test cases that trivially return 400 user errors would not be helpful for our goals.

The integration of our approach into existing fuzzers is as follows. We apply our security validation as a *post-processing* phase. We let a fuzzer run for the amount of time specified by the users. In academic experiments, this is typically 1 h (Golmohammadi et al., 2023). Within 1-hour, a fuzzer could evaluate hundreds of thousands of HTTP calls. However, most likely no human test engineer would be interested in analyzing and using test suites with hundreds of thousands of test cases (Arcuri et al., 2025a). It is simply not feasible. As such, fuzzers typically provide as output a minimized test suite, aimed at maximizing different criteria. These could be about covering different black-box aspects of the API (Martin-Lopez et al., 2019) (e.g., based on the OpenAPI schema, like covering different HTTP status codes for each defined endpoint), or code coverage metrics in white-box testing (Arcuri, 2019).

```

1 /**
2  * Calls:
3  * 1 - (200) POST:/api/sqli/body/vulnerable
4  * 2 - (200) POST:/api/sqli/body/vulnerable
5  * Found 1 potential fault of type-code 200
6  */
7 @Test @Timeout(60)
8 fun test_3_postOnVulnerableVulnerableToSQLInjection() {
9
10
11     // res_0_ms stores the start time in milliseconds
12     val res_0_ms = System.currentTimeMillis()
13
14     given().accept("/*/*")
15         .header("x-EMextraHeader123", "")
16         .contentType("application/json")
17         .body(" { " +
18             "  \"password\": \"_EM_110_XYZ_\", " +
19             "  \"username\": \"_EM_111_XYZ_\" " +
20             " } ")
21         .post("${baseUrlOfSut}/api/sqli/body/vulnerable?
22             EMextraParam123=_EM_112_XYZ_")
23         .then()
24         .statusCode(200)
25         .assertThat()
26         .contentType("text/plain")
27         .body(containsString("MATCHED: 0"))
28
29     // res_2_ms stores the total execution time in milliseconds
30     val res_2_ms = System.currentTimeMillis() - res_0_ms
31
32     // Note: No SQL Injection vulnerability detected in this call.
33     // Expected response time (sqliBaselineMaxResponseTimeMs) should be less
34     // than 2000 ms.
35     assertTrue(res_2_ms < 2000)
36
37
38     // res_3_ms stores the start time in milliseconds
39     val res_3_ms = System.currentTimeMillis()
40
41     // Fault200. SQL Injection (SQLi).
42     given().accept("/*/*")
43         .header("x-EMextraHeader123", "")
44         .contentType("application/json")
45         .body(" { " +
46             "  \"password\": \"_EM_2_XYZ_\u0027 OR SLEEP(5.00)-- -\", " +
47             "  \"username\": \"_EM_3_XYZ_\u0027 OR SLEEP(5.00)-- -\" " +
48             " } ")
49         .post("${baseUrlOfSut}/api/sqli/body/vulnerable?
50             EMextraParam123=_EM_4_XYZ_%27OR+SLEEP%285.00%29--+--")
51         .then()
52         .statusCode(200)
53         .assertThat()
54         .contentType("text/plain")
55         .body(containsString("MATCHED: 0"))
56
57     // res_5_ms stores the total execution time in milliseconds
58     val res_5_ms = System.currentTimeMillis() - res_3_ms
59
60     // Note: SQL Injection vulnerability detected in this call. Expected
61     // response time (sqliInjectedSleepDurationMs) should be greater than
62     // 5000 ms.
63     assertTrue(res_5_ms > 5000)
64 }

```

Fig. 8. Example of generated test for an artificial API in which a SQL Injection vulnerability is detected.

After the fuzzing session is over, the fuzzer would have generated a set T of test cases. These tests are then used as a base to create new test scenarios, as previously described in Section 3. As such, how these T tests are created is not so important, and any state-of-the-art fuzzer could be used, as long as it can generate tests using authentication information. Furthermore, technically there would not even be the need of a fuzzer, as existing manually written tests (if any) could be used as starting point T . The technical challenge there would then be how to effectively *carve* those tests to enable their manipulation and modifications for generating new test cases.

For our experiments, we used EvoMASTER, as it fits those constraints. However, one limitation is that it was not good at generating tests

in which 403 unauthorized accesses were detected. Given a set of authentication information as input to the fuzzer (besides a copy of the OpenAPI schema for the tested API), EvoMASTER can generate tests with and without authentication information. However, calls with different users on the same resources (which could lead to 403 errors) were not handled. As such, we needed to make sure to try to create such kind of tests before applying our novel oracles, as some of those rely on existing tests with 403 responses to create the needed test scenarios.

For each verb/path $V : X$ in the OpenAPI schema, we check if we have any test case generated in T in which such a call returns a 403 (unauthorized). If not, we try to create a new test for it and add it to T . If there is no call in T with a 401 (not authenticated), then most likely

```

1 /**
2 * Calls:
3 * 1 - (200) POST:/api/stored/json/guestbook
4 * 2 - (200) GET:/api/stored/json/guestbook
5 * Found 1 potential fault of type-code 201
6 */
7 @Test @Timeout(60)
8 fun test_3_getOnGuestbookVulnerableToXSS() {
9
10     given().accept("application/json")
11         .header("x-EMextraHeader123", "<img src=x onerror=alert('XSS
12         '>')>")
13         .post("${baseUrlOfSut}/api/stored/json/guestbook?" +
14             "name=%3Cimg+src%3Dx+onerror%3Dalert%28%27XSS%27%29%3E&"
15             +
16             "entry=%3Cimg+src%3Dx+onerror%3Dalert%28%27XSS%27%29%3E&"
17             +
18             "EMextraParam123=%3Cimg+src%3Dx+onerror%3Dalert%28%27XSS
19             %27%29%3E")
20         .then()
21         .statusCode(200)
22         .assertThat()
23         .contentType("application/json")
24         .body("message", containsString("Guestbook Entry Stored!
25         Thank you for signing our guestbook!"))
26         .body("success", equalTo(true))
27
28     // Fault201. Cross-Site Scripting (XSS).
29     given().accept("application/json")
30         .header("x-EMextraHeader123", "<img src=x onerror=alert('XSS
31         '>')>")
32         .get("${baseUrlOfSut}/api/stored/json/guestbook?
33         EMextraParam123=%3Cimg+src%3Dx+onerror%3Dalert%28%27XSS%27%29%3E")
34         .then()
35         .statusCode(200)
36         .assertThat()
37         .contentType("application/json")
38         .body("entries.size()", equalTo(1))
39         .body("entries[0].name", containsString("<img src=x
40         onerror=alert('XSS')>"))
41         .body("entries[0].entry", containsString("<img src=x
42         onerror=alert('XSS')>"))
43 }

```

Fig. 9. Example of a generated test for an artificial API in which a stored XSS potential vulnerability is detected.

there is no authentication mechanism on that endpoint, and it can be skipped. Otherwise, we then check if there is any test T_a for which a successful 2xx response is obtained with any authenticated user A . If so, we make a copy C_a in which all calls after the target are sliced away. Then, for each other defined user B , we create a copy C_b of C_a , and we duplicate the last call in C_b by replacing A with B . For example, if we have C_a being:

```

1 (A) POST /data -> 201
2 (A) GET /data/42 -> 200

```

then C_b would be:

```

1 (A) POST /data -> 201
2 (A) GET /data/42 -> 200
3 (B) GET /data/42 -> 403

```

If executing C_b returns a 403 from the last call, then we are done for $V : X$, and C_b can be added to T . Otherwise, we repeat this process for the other users (if there is more than two) and for any other tests T_a that meet those criteria.

The number of new test cases we generate to evaluate security properties in our post-processing phase is linearly dependent on the number of endpoints N in the tested API, i.e., its complexity is $O(N)$. Typically, these are at most a few hundreds. As such, creating and evaluating these security tests takes just a few seconds or minutes, depending on the tested API. In the context of fuzzing an API for hours (e.g., between 1 and 24 h), in most cases the overhead of our novel techniques is practically negligible, as we will show in our empirical study in Section 6.6.

As for any dynamic technique in which test cases are generated and evaluated against a live version of an API, rate limiters might negatively impact the results. However, this would apply to any existing REST

API fuzzer, independently of the novel oracles presented in this paper. As this is a common issue in practice, state-of-the-art tools such as EvoMASTER already handle such type of issue. For example, EvoMASTER automatically handles 429 responses (Too Many Requests), by automatically slowing down the execution of HTTP calls based on the returned wait times. Furthermore, to prevent hitting such rate limiters in the first place, it is possible to self-constrain the amount of requests per minute by using options such as `--ratePerMinute`.

While a hacker targeting a remote API would be impacted by rate limiters for the API calls, this is less of a problem for testers and engineers evaluating their own APIs. In those cases, rate limiters might be disabled (e.g., a developer running the API on their working laptop), or adequately configured when the API is run in a testing environment (in contrast to production environment). This is also the case for different kinds of testing activities, such as stress and performance testing.

6. Empirical study

In this paper, we carried out an empirical study aimed at answering the following research questions:

RQ1: Are our novel techniques able to identify injected faults in artificial examples?

RQ2: Are our novel techniques able to identify faults in example APIs used in the security literature?

RQ3: What security faults can be found in existing real-world APIs?

RQ4: What is the overhead cost of running our novel oracles?

Table 1

Statistics of the employed REST APIs in our empirical study for black-box testing, including number of source files, numbers of lines of code (LOCs), the number of HTTP endpoints, and their identifier on GitHub.

SUT	#SourceFiles	#LOCs	#Endpoints	GitHub
<i>capital</i>	103	4481	21	<i>Checkmarx/capital</i>
<i>crapi</i>	564	129 026	44	<i>OWASP/crAPI</i>
<i>damn-vulnerable-rest</i>	75	4470	21	<i>theowni/Damn-Vulnerable-RESTaurant-API-Game</i>
<i>dvapi</i>	13	1211	16	<i>payatu/DVAPI</i>
<i>dvws-node</i>	27	3083	31	<i>snoopysecurity/dvws-node</i>
<i>vampi</i>	11	520	14	<i>erev0s/VAmPI</i>
<i>vulnerable-rest-api</i>	30	837	19	<i>bnematzadeh/vulnerable-rest-api</i>
<i>webgoat</i>	401	25 972	201	<i>WebGoat/WebGoat</i>
Total 8	1224	169 600	367	

6.1. API selection

To answer **RQ1**, we created nine small APIs, using Kotlin and SpringBoot. In each one, we injected a security fault for each of our seven novel oracles and the two injection cases. These APIs are simple, with no complex input constraints, or code execution flow depending on database state. Faults might not be detected if the code in which they reside is never executed due to the complexity of generating the right data. Our goal here in **RQ1** is to study the feasibility of our novel techniques without confounding factors.

For reasons of space, we do not provide here full code of these nine example APIs. They are provided open-source with our extension to EvoMASTER. The examples shown in Section 3 were based on these artificial APIs, like the generated tests shown in Figs. 1–9.

To answer **RQ2**, we looked at the security research literature for existing APIs used for demonstrating different types of security vulnerabilities. After a search in popular resources such as OWASP, we selected eight widely used open-source APIs. Table 1 shows some statistics on these eight APIs, including links to their repository on GitHub. All these APIs are run through Docker Compose, where all of their dependencies are started as well (e.g., databases). Note that *crapi* is a microservice running different services. All are run in our experiments. We fuzz the whole application through its API Gateway.

Note that these APIs are deliberately vulnerable, with several different types of injected security faults. All these APIs require authentication information (e.g., valid usernames/passwords for some existing users). When the applications start, such info is set in their databases. Authentication information for the fuzzer is provided in YAML files, using the Web Fuzzing Commons (WFC) format Sahin et al. (2025).

To answer **RQ3**, we selected an existing corpus of REST APIs used in several studies in API fuzzing research, called WFD (Sahin et al., 2025), formerly known as EMB (Arcuri et al., 2023b), which has been available open-source and extended throughout the years since 2017. We selected *all* APIs in its most recent version.¹⁵ Table 2 shows some statistics on these 36 APIs. WFD is a variegated corpus, including small example APIs, as well as large real-world APIs (e.g., coming from public administrations from different countries such as Germany, Norway, Thailand and the United Kingdom). For the APIs that need it, authentication information in WFD is provided in YAML files, using the WFC format Sahin et al. (2025), like for the experiments for **RQ2**.

Note that security faults can be found only if they are present. Access policy violations cannot be found if an API has no authentication mechanism. Likewise, no SQL Injection can be found if the API does not use any SQL database. We have not injected any artificial fault in any of these APIs. In our experiments, these 36 APIs are used as they are.

To answer **RQ4**, we use the data from the experiments for **RQ3**, where we measure the execution time of our security testing phase after the fuzzing phase.

In total, for our experiments in this paper we used $9 + 8 + 36 - 1 = 52$ distinct APIs, for a total of more than 800 thousand lines of code (for

business logic, not including third-party libraries), and more than 1600 endpoints. Note that *webgoat* is used twice, both for **RQ2** and **RQ3**. The API *webgoat* is a deliberately insecure application, built and maintained by OWASP. As such, it is part of **RQ2**. However, as it is written in Java, it was added to WFD in the past. Therefore, we run it as well in **RQ3**. Note that, as discussed in the next section, experiments for **RQ2** and **RQ3** are run differently (i.e., *black-box* vs. *white-box* testing). As such, reusing a single API twice should not be problematic.

6.2. Experiments setup

The 9 artificial examples used for answering **RQ1** are small and simple. As such, experiments were run with our extension of EvoMASTER for just 1 min, 10 times, for a total of 90 min.

Experiments for **RQ2** were run using *black-box* mode of EvoMASTER. On each of these 8 APIs, EvoMASTER was run for 1 h. Experiments were repeated 10 times per API, for a total of 80 h.

As all the APIs used for **RQ3** are written in Java or Kotlin, to answer this research question we use the *white-box* mode of EvoMASTER. We use the same time budget and repetition settings as in **RQ2**, i.e., EvoMASTER was run for 1 h, with each experiment repeated 10 times. This resulted in a duration of 360 h.

Our novel oracles can work both for black-box and white-box testing. We use different settings in **RQ2** and **RQ3** to show this being indeed the case. In total, our experiments took more than 440 h, i.e., more than 18 days if run in sequence.

Answering **RQ4** did not require running any further experiment, as based on the same data for the other RQs.

6.3. Results for RQ1

In every single experiment for **RQ1**, the injected faults were correctly identified. When there are no complex input constraints that prevent reaching the execution of the faulty code, our novel oracles are able to correctly identify all the security vulnerabilities that we have manually injected.

EvoMASTER has an advanced and sophisticated system of end-to-end tests to verify its correctness (Arcuri et al., 2023a). There are several artificial APIs in which EvoMASTER is automatically run on, and the generated tests are compiled and run, verifying different properties (e.g., if injected faults are properly identified). These checks on these artificial APIs are run as JUnit tests as part of EvoMASTER's own continuous integration (CI) system (Arcuri et al., 2023a) (e.g., on GitHub Actions). These nine new artificial examples are now part of these regression test suites, executed in CI at each new code commit of EvoMASTER, as they can be reliably handled by EvoMASTER.

RQ1: On simple APIs with injected faults, our techniques are able to reliably identify all those 9 types of security vulnerabilities.

¹⁵ <https://github.com/WebFuzzing/Dataset>

Table 2

Statistics of the employed 36 REST APIs in our empirical study for white-box testing, including number of source files, numbers of lines of code (LOCs), the number of HTTP endpoints, whether they require authentication, and the databases they use (if any).

SUT	#SourceFiles	#LOCs	#Endpoints	Auth	Databases
<i>bibliothek</i>	33	2176	8		MongoDB
<i>blogapi</i>	89	4787	52	✓	MySQL
<i>catwatch</i>	106	9636	14		H2
<i>cwa-verification</i>	47	3955	5		H2
<i>erc20-rest-service</i>	7	1378	13		
<i>familie-ba-sak</i>	1089	143 556	183	✓	PostgreSQL
<i>features-service</i>	39	2275	18		H2
<i>genome-nexus</i>	405	30 004	23		MongoDB
<i>gestaohospital</i>	33	3506	20		MongoDB
<i>http-patch-spring</i>	30	1450	6		
<i>languagetool</i>	1385	174 781	2		
<i>market</i>	124	9861	13	✓	H2
<i>microcks</i>	471	66 186	88	✓	MongoDB
<i>ocvn</i>	526	45 521	258	✓	H2;MongoDB
<i>ohsome-api</i>	87	14 166	134		OSHDB
<i>pay-publicapi</i>	377	34 576	10	✓	Redis
<i>person-controller</i>	16	1112	12		MongoDB
<i>proxyprint</i>	73	8338	74	✓	H2
<i>quartz-manager</i>	129	5068	11	✓	
<i>reservations-api</i>	39	1853	7	✓	MongoDB
<i>rest-ncs</i>	9	605	6		
<i>rest-news</i>	11	857	7		H2
<i>rest-scs</i>	13	862	11		
<i>restcountries</i>	24	1977	22		
<i>scout-api</i>	93	9736	49	✓	H2
<i>session-service</i>	15	1471	8		MongoDB
<i>spring-actuator-demo</i>	5	117	2	✓	
<i>spring-batch-rest</i>	65	3668	5		
<i>spring-ecommerce</i>	58	2223	26	✓	MongoDB;Redis;Elasticsearch
<i>spring-rest-example</i>	32	1426	9		MySQL
<i>swagger-petstore</i>	23	1631	19		
<i>tiltaksgjennomforing</i>	472	27 316	79	✓	PostgreSQL
<i>tracking-system</i>	87	5947	67	✓	H2
<i>user-management</i>	69	4274	21		MySQL
<i>webgoat</i>	355	27 638	204	✓	H2
<i>youtube-mock</i>	29	3229	1		
Total 36	6465	657 162	1487	15	25

Table 3

Detected faults on the 8 vulnerable APIs.

SUT	#Endp	F100	F200	F201	F204	F205	F206	F902	F903
<i>capital</i>	21			1.4	0.9		0.8		
<i>crapi</i>	44	14.0				32.5			5.0
<i>damn-vulnerable-rest</i>	21	5.8		2.9	0.5	2.0	0.1		
<i>dvapi</i>	16	3.0							10.0
<i>dvws-node</i>	31	3.5	0.6	3.6					
<i>vampi</i>	14	4.6		0.3				4.0	5.0
<i>vulnerable-rest-api</i>	19	11.0		1.0		1.0			
<i>webgoat</i>	201	33.0		11.4		5.0		25.0	3.0
Mean	46	9.4	0.1	2.6	0.2	5.1	0.1	3.6	2.9
Median	21	5.2	0.0	1.2	0.0	0.5	0.0	0.0	1.5
Sum	367	74.9	0.6	20.6	1.4	40.5	0.9	29.0	23.0

6.4. Results for RQ2

Table 3 shows the results for RQ2 on eight vulnerable APIs. Besides the results for our novel oracles, we also report the number of faults related to HTTP 500 status codes, which is the most common oracle in the REST API fuzzing literature. EvoMASTER can also detect other types of functional faults (e.g., schema mismatches) and SSRF, but those are not discussed and presented in this paper. Faults are based on the endpoint calls that led to trigger them. For example, if an API has n endpoints, there could be detected at most n distinct faults related to HTTP 500. It could be possible that the code of an endpoint has more than one fault that could be triggered differently, or that the same one single fault can lead to more than one endpoint to fail. Counting based on number of failing endpoints is a common, reasonable compromise when counting the number of faults, especially in black-box testing, where a root-cause analysis cannot be easily automated.

To simplify the discussion and references to these faults, each one has a unique label, based on the labeling system defined in WFC (Sahin et al., 2025). In particular, we have:

- F100: HTTP Status 500
- F200: SQL Injection (Section 4.1)
- F201: Cross Site Scripting (XSS) (Section 4.2)
- F204: Existence Leakage (Section 3.2)
- F205: Not Recognized Authentication (Section 3.1)
- F206: Missed Authorization Checks (Section 3.3)
- F900: Ignore Anonymous (Section 3.5)
- F901: Anonymous Modifications (Section 3.4)
- F902: Leaked Stack Trace (Section 3.6)
- F903: Hidden Accessible (Section 3.7)

Table 4
Detected faults on the 36 APIs from WFD.

SUT	#Endp	F100	F201	F204	F205	F900	F901	F902	F903
<i>bibliothek</i>	8	2.7							26.0
<i>blogapi</i>	52	28.5	0.1		0.5				7.0
<i>catwatch</i>	14	6.4				6.2		2.5	
<i>cwa-verification</i>	5	5.0							
<i>erc20-rest-service</i>	13	12.0							
<i>familie-ba-sak</i>	183	151.2	2.0		12.0	4.6			
<i>features-service</i>	18	12.5					1.0	12.3	
<i>genome-nexus</i>	23		5.0						2.0
<i>gestaohospital</i>	20	1.9	2.2				3.6		6.0
<i>http-patch-spring</i>	6		0.5				1.2		
<i>languagetool</i>	2	1.0							
<i>market</i>	13	5.3							
<i>microcks</i>	88	10.2	7.9						14.0
<i>ocvn</i>	258	194.3						82.9	
<i>ohsome-api</i>	134	1.6							
<i>pay-publicapi</i>	10	10.0							
<i>person-controller</i>	12	11.0						5.0	2.0
<i>proxyprint</i>	74	33.9		2.0	3.7			8.7	
<i>quartz-manager</i>	11	5.9							
<i>reservations-api</i>	7	3.8			0.1				2.0
<i>rest-ncs</i>	6								
<i>rest-news</i>	7	2.9					2.4		
<i>rest-scs</i>	11								
<i>restcountries</i>	22	1.0							
<i>scout-api</i>	49	15.2	0.3						
<i>session-service</i>	8	3.6							2.0
<i>spring-actuator-demo</i>	2		1.0						
<i>spring-batch-rest</i>	5	1.8	1.0						
<i>spring-ecommerce</i>	27	15.8			3.1	0.3			
<i>spring-rest-example</i>	9	1.0	0.4				4.0		
<i>swagger-petstore</i>	19	9.2	3.8				3.0		
<i>tiltaksgjennomforing</i>	79	28.7	1.0						8.0
<i>tracking-system</i>	67		10.6				1.9		21.0
<i>user-management</i>	21	6.7	5.2				3.7		4.0
<i>webgoat</i>	204	5.0						4.0	2.0
<i>youtube-mock</i>	1								
Mean	41	16.3	1.1	0.1	0.5	0.3	0.6	3.2	2.7
Median	14	5.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Sum	1488	588.1	41.0	2.0	19.4	11.1	20.8	115.4	96.0

Note that codes F1xx are for functional oracles, whereas F2xx are for security faults. Codes in the group 9xx are for work-in-progress codes not part of WFC yet (Sahin et al., 2025). The use of unique, stable codes is meant for simplifying automated comparisons of fuzzers that use the WFC convention. Table 3 shows only data for which faults were found. If a fault type was never found in any of the 10 runs on any of the 8 APIs, such data is not present in the table (to avoid cluttering tables with 0-value columns).

In these experiments, tens of security faults were found, for 8 out of 10 different kinds. No fault for F900 (Ignore Anonymous) and F901 (Anonymous Modifications) were found. Faults can only be found if they exist. We have not manually reviewed the whole code and documentation of those 8 APIs to see if there are any other faults that could have been found with our oracles, or if what found is all that is there. Still, the fact that our novel techniques could generate test cases that can detect these faults show that they can be of practical value. However, we have manually reviewed the generated tests, to see if indeed the faults they detect are false positives. In our manual review we did not find any false positive.

RQ2: Our novel oracles were effective at detecting tens of faults in vulnerable APIs used as security examples.

6.5. Results for RQ3

When it comes to white-box testing of the APIs in WFD, in Table 4 we can see that security faults are found in most of the APIs, with hundreds of distinct cases. However, no fault of types F200 (SQL Injection) and F206 (Missed Authorization Checks) were found. These

are among the most critical types of faults that we can detect with our novel techniques.

Table 4 shows that, on the 36 analyzed APIs, in total our oracles detected more than 300 distinct faults (although not all were found in all runs). This provides evidence for the practical value of our novel techniques.

An API like *familie-ba-sak* is large and complex (Table 2). It is made by the public administration in Norway, dealing with “child benefit case processing”, used by millions of people.¹⁶ Finding a Missed-Check or SQL Injection would have rather been unsettling, and we are glad none was found. (If any was found, we would have contacted the Norwegian authorities before making this paper public).

Interestingly, there were several cases of *Not Recognized Authentication* (Section 3.1). Fig. 10 shows an example of generated test detecting a *Not Recognized Authentication* (Section 3.1) fault. This automatically generated test case starts with 2 calls to the authentication service, to get dynamic authentication tokens for the users *Veileder* (Line 8) and *TaskRunner* (Line 15). The first call on */api/task/logg/710* with *Veileder* shows that such endpoint requires authentication/authorization, as a 401 status code is returned (Line 24). A second call on the same endpoint with *TaskRunner* gets a 200 status code (Line 33). This provides evidence that the credentials for *TaskRunner* are not wrongly set. However, a call with the same user on */api/ekstern/pensjon/hent-barnetrygd* returns a 401 (Line 52), which would imply the user is not recognized. Based on previous calls, this is wrong. Note that, even if access tokens might have a short

¹⁶ <https://github.com/navikt/familie-ba-sak>

```

1 @Test @Timeout(60)
2 public void test_138_postOnHent_barnetrygdAuthenticatedButWronglyToldNot
   () throws Exception {
3
4     final String token_Veileder = given()
5         .contentType("application/x-www-form-urlencoded")
6         .body("name=Veileder&grant_type=client_credentials&code=foo&
           client_id=foo&client_secret=secret")
7         .post("http://localhost:10512/azuread/token")
8         .then().extract().response().path("access_token");
9
10    final String auth_Veileder = "Bearer " + token_Veileder;
11    final String token_TaskRunner = given()
12        .contentType("application/x-www-form-urlencoded")
13        .body("name=TaskRunner&grant_type=client_credentials&code=foo
           &client_id=foo&client_secret=secret")
14        .post("http://localhost:10512/azuread/token")
15        .then().extract().response().path("access_token");
16
17    final String auth_TaskRunner = "Bearer " + token_TaskRunner;
18
19    given().accept("/*/*")
20        .header("x-EMextraHeader123", "_EM_14362_XYZ_")
21        .header("Authorization", auth_Veileder) // Veileder
22        .get(baseUrlOfSut + "/api/task/logg/710?page=352")
23        .then()
24        .statusCode(401)
25        .assertThat()
26        .contentType("text/html");
27
28    given().accept("/*/*")
29        .header("x-EMextraHeader123", "42")
30        .header("Authorization", auth_TaskRunner) // TaskRunner
31        .get(baseUrlOfSut + "/api/task/logg/718")
32        .then()
33        .statusCode(200)
34        .assertThat()
35        .contentType("application/json")
36        .body("data", nullValue())
37        .body("status", containsString("FEILET"))
38        .body("melding", containsString("Henting av tasklogg feilet
           ."))
39        .body("frontendFeilmelding", containsString("Henting av
           tasklogg feilet."));
40
41    // Fault205. Wrongly Not Recognized as Authenticated.
42    given().accept("application/json")
43        .header("x-EMextraHeader123", "42")
44        .header("Authorization", auth_TaskRunner) // TaskRunner
45        .contentType("application/json")
46        .body("{ \" +
47            \" \"ident\": \"_EM_14364_XYZ_\", \" +
48            \" \"fraDato\": \"2020-12-01\" \" +
49            \" } \"")
50        .post(baseUrlOfSut + "/api/ekstern/pensjon/hent-barnetrygd?
           EMextraParam123=_EM_14366_XYZ_")
51        .then()
52        .statusCode(401)
53        .assertThat()
54        .contentType("application/json")
55        .body("servlet", containsString("dispatcherServlet"))
56        .body("message", containsString("Pensjon tjeneste kan kun
           kalles av pensjon eller innlogget bruker med FORVALTER rolle"))
57        .body("url", containsString("/api/ekstern/pensjon/hent-
           barnetrygd"))
58        .body("status", containsString("401"));
59 }

```

Fig. 10. Snippet example of generated test case in which a *Not Recognized Authentication* (Section 3.1) fault is identified in *familie-ba-sak*. Due to reason of space, the generated test's JavaDoc is not shown.

lifespan, they are dynamically retrieved in the same test case, and, as such, after a few (milli) seconds they should still be valid. Interestingly, the error message (in Norwegian at Line 56) seems to support the fact that this is a bug (i.e., a 403 should have been returned), as it states, translated with Google Translate: "Pension service can only be called by pension or logged in user with ADMINISTRATOR role".

Not all faults have the same severity. For example, a 500 status does not really mean that the API crashed. If it crashed, then it would not be able to send back an HTTP response with status 500, and the TCP socket would have been closed. Typically, when an exception is thrown, then the framework (e.g., Spring for Java) running the API catches the

exception, and sends back an error response. The API will keep running. If the crash is due to invalid input, then the fault is related to improper input validation, and rather a 4xx status error (indicating an user error) should have been returned. A typical example is assuming a requested resource to exist, and crashing on a null-pointer exception instead of returning a 404 not found user error status (Marculescu et al., 2022). This is still a fault that needs to be fixed, but it is not as critical as it might look at first sight. In this paper, we have proposed 7 novel oracles, but we cannot say how critical they are for certain, unless empirical studies or interviews/surveys with practitioners were carried out to assess their assumed importance. However, we argue that a security vulnerability

```

1 /**
2 * Calls:
3 * 1 - (200) OPTIONS:/api/users/checkEmailAvailability
4 * 2 - (415) PUT:/api/users/checkEmailAvailability
5 * Found 3 potential faults. Type-codes: 101, 903
6 */
7 @Test(timeout = 60000)
8 public void test_46_putOnCheckEmailAvailabilityShowsFaults_101_903()
9     throws Exception {
10
11     final String token_admin = given()
12         .contentType("application/json")
13         .body("{ \" " +
14             " \"usernameOrEmail\": \"admin\", \" " +
15             " \"password\": \"bar123\" \" " +
16             " } ")
17         .post(baseUrlOfSut + "/api/auth/signin")
18         .then().extract().response().path("accessToken");
19
20     final String auth_admin = "Bearer " + token_admin;
21
22     // Fault101. Received A Response From API With A Structure/Data That
23     // Is Not Matching Its Schema. Type: validation.request.operation.
24     // notAllowed OPTIONS operation not allowed on path '/api/users/
25     // checkEmailAvailability'.
26     given().accept("/*/*")
27         .header("Authorization", auth_admin) // admin
28         .options(baseUrlOfSut + "/api/users/checkEmailAvailability")
29         .then()
30         .statusCode(200)
31         .assertThat()
32         // .header("Allow", "HEAD,DELETE,GET,OPTIONS,PUT")
33         .body(isEmptyOrNullString());
34
35     // Fault101. Received A Response From API With A Structure/Data That
36     // Is Not Matching Its Schema. Type: validation.response.body.unexpected
37     // No response body is expected but one was found.
38     // Fault903. Hidden Accessible Endpoint.
39     given().accept("/*/*")
40         .header("Authorization", auth_admin) // admin
41         .put(baseUrlOfSut + "/api/users/checkEmailAvailability")
42         .then()
43         .statusCode(415)
44         .assertThat()
45         .contentType("application/json")
46         .body("status", numberMatches(415))
47         .body("error", containsString("Unsupported Media Type"))
48         .body("message", containsString("Content type 'application/
49 x-www-form-urlencoded; charset=UTF-8' not supported"))
50         .body("path", containsString("/api/users/
51 checkEmailAvailability"));
52 }

```

Fig. 11. Example of generated test for *blogapi* finding a F903 *Hidden Accessible Endpoint* fault.

that leads to unauthorized access of resources (e.g., *Missed-Check*) is likely much more important than wrongly returning a 500 status code instead of a 404.

In the data generated in our experiments, there are hundreds of interesting cases that could be worth discussing. For reasons of space, we cannot discuss all of them. We have done a manual review of all detected faults, and there was no false positive. In all cases, what detected was indeed a fault that requires fixing. However, we noticed that, in a few cases, when our novel oracles detected faults, the problem was not due to a security vulnerability, but rather due to severe faults related to HTTP semantics. To clarify, we will discuss here some of these cases in more details.

Figs. 11 and 12 show two distinct examples of F903 *Hidden Accessible Endpoint* faults, in the *blogapi* and *bibliotek* APIs. After a manual investigation of these APIs, there is no hidden accessible endpoint in these APIs. There are two other problems though. First, the OPTIONS is returning what could be considered as “garbage”, claiming in the *Allow* header the presence of endpoints that do not exist. Second, calling those endpoints does not return a 405 (or 501), as expected by HTTP specifications. They rather return a 415 (for *blogapi*) or 404 (*bibliotek*). In either case, these response statuses make no sense according to the semantics of HTTP. This is not a fault in the implementation of the business logic of these APIs, but rather a serious fault or misconfiguration in their employed HTTP frameworks.

Figs. 13 and 14 show two distinct examples of F900 *Ignore Anonymous* faults, in the *catwatch* and *familie-ba-sak* APIs. In theory, this is a potentially catastrophic vulnerability. The API *catwatch* has no defined authentication information in WFD (recall Table 2). A call to the endpoint GET:/config without any authentication info returns a 200 with some data. However, a non-authenticated call with the query parameter access_token=cv0YB8K5cuf (Line 15) leads to return a 403. Without an in-depth analysis of the semantics of this API, it is unclear if this is a major security vulnerability, or misuse of HTTP status codes. Either way, it is problematic.

The case of *familie-ba-sak* in Fig. 14 is easier to analyze, but potentially more disturbing. An authenticated user “Veileder” makes a POST to an endpoint, successfully returning a 200 status code (Line 11). A different user “Forvalter”, with different permission rights, is blocked from doing the same operation, and obtains a 403 (Line 29). Finally, a call without any authentication information is successful with a 200 status code (Line 42). Without looking at the response, this could be a major security issue. However, FEILET in Norwegian means “the error”, and “Fant ikke aktiv ident for aktør” means “No active ident found for actor”. In other words, the call fails, but the API decides to return a 200 success status code, which seems completely wrong in this context.

The case of Fig. 14 is interesting because, without a natural-language processing (NLP) of the text of the responses, it could be

```

1 /**
2 * Calls:
3 * 1 - (404) OPTIONS:/v2/projects/{project}
4 * 2 - (404) POST:/v2/projects/{project}
5 * Found 1 potential fault of type-code 903
6 */
7 @Test @Timeout(60)
8 public void test_6_postOnProjectHiddenAccessible() throws Exception {
9
10     given().accept("*/*")
11         .options(baseUrlOfSut + "/v2/projects/omm_postfix")
12         .then()
13         .statusCode(404)
14         .assertThat()
15         // .header("Allow", "HEAD,DELETE,POST,GET,OPTIONS,PUT,PATCH")
16         .contentType("application/json")
17         .body("'error'", containsString("Endpoint not found.));
18
19     // Fault903. Hidden Accessible Endpoint.
20     given().accept("*/*")
21         .post(baseUrlOfSut + "/v2/projects/omm_postfix")
22         .then()
23         .statusCode(404)
24         .assertThat()
25         .contentType("application/json")
26         .body("'error'", containsString("Endpoint not found.));
27 }

```

Fig. 12. Example of generated test for *bibliotek* finding a F903 Hidden Accessible Endpoint fault.

```

1 /**
2 * Calls:
3 * 1 - (403) GET:/config
4 * 2 - (200) GET:/config
5 * Found 3 potential faults. Type-codes: 101, 900
6 */
7 @Test(timeout = 60000)
8 public void test_69_getOnConfigShowsFaults_101_900() throws Exception {
9
10     // Fault101. Received A Response From API With A Structure/Data That Is Not Matching Its Schema.
11     // Type: validation.response.body.unexpected No response body is expected but one was found.
12     given().accept("application/json")
13         .header("x-EMextraHeader123", "")
14         .header("X-Organizations", "urLJpDVv")
15         .get(baseUrlOfSut + "/config?" +
16             "access_token=cv0YB8K5cuf&" +
17             "q=HXDhFkqk")
18         .then()
19         .statusCode(403)
20         .assertThat()
21         .contentType("application/json")
22         .body("'error'", containsString("access_denied"))
23         .body("'error_description'", containsString("Unable to obtain a new access token
24             for resource &#39;null&#39;. The provider manager is not configured to support it.));
25
26     // Fault101. Received A Response From API With A Structure/Data That Is Not Matching Its Schema.
27     // Type: validation.response.body.schema.type [Path '/github.login'] Instance type (null) does not
28     // match any allowed primitive type (allowed: ["string"])
29     // Fault900. A Protected Resource Is Accessible Without Providing Any Authentication.
30     given().accept("application/json")
31         .header("x-EMextraHeader123", "")
32         .get(baseUrlOfSut + "/config?" +
33             "q=_EM_6668_XYZ_&" +
34             "organizations=LVszttcjhEvq0L1&" +
35             "limit=tawdhQd0NfwEwo")
36         .then()
37         .statusCode(200)
38         .assertThat()
39         .contentType("application/json")
40         .body("'cache.path'", containsString("D:\\WORK\\EXPERIMENTS\\2026-security\\
41             wb_0_9/temp/tmp_catwatch/cache_10212"))
42         .body("'cache.size'", containsString("50"))
43         .body("'endpoints.enabled'", containsString("false"))
44         .body("'endpoints.health.enabled'", containsString("true"))
45         .body("'github.login'", nullValue())
46         .body("'organization.list'", containsString("zalando,zalando-stups,zalando-
47             incubator"))
48         .body("'schedule'", containsString("0 1 8 * * *"))
49         .body("'scoring.project'", containsString("function(project) {var
50             daysSinceLastPush = 0;if (project.lastPushed) {var tokens = project.lastPushed.split("\\ ");var
51             day = tokens[2];var month = tokens[1];var year = tokens[5];var lastPushedDate = new Date(day + ' '
52             + month + ' ' + year);var millisInDay = 86400000;daysSinceLastPush = Math.floor((new Date() -
53             lastPushedDate) / millisInDay);}var maintainersPenalty = 0;if (project.maintainers.length < 2) {
54             maintainersPenalty = 100;}return project.starsCount * 3 + project.forksCount * 2 + project.
55             contributorsCount * 5 - daysSinceLastPush - maintainersPenalty}"));
56         .body("'spring.database.driverClassName'", nullValue())
57         .body("'spring.datasource.platform'", nullValue())
58         .body("'spring.datasource.username'", containsString("sa"))
59         .body("'spring.jpa.database'", nullValue())
60         .body("'spring.jpa.hibernate.ddl-auto'", containsString("update"));
61 }

```

Fig. 13. Example of generated test for *catwatch* finding a F900 Ignore Anonymous fault.

```

1  ...
2  given().accept("*/")
3      .header("x-EMextraHeader123", "")
4      .header("Authorization", auth_Veileder) // Veileder
5      .contentType("application/json")
6      .body(" { " +
7             "   \"ident\": \"\" " +
8             " } ")
9      .post(baseUrlOfSut + "/testverktøy/vedtak-om-overgangsstnad")
10     .then()
11     .statusCode(200)
12     .assertThat()
13     .contentType("application/json")
14     .body("data", nullValue())
15     .body("status", containsString("FEILET"))
16     .body("melding", containsString("Fant ikke aktiv ident for aktør"))
17     .body("frontendFeilmelding", nullValue())
18     .body("stacktrace", nullValue());
19
20  given().accept("*/")
21      .header("x-EMextraHeader123", "")
22      .header("Authorization", auth_Forvalter) // Forvalter
23      .contentType("application/json")
24      .body(" { " +
25             "   \"ident\": \"\" " +
26             " } ")
27      .post(baseUrlOfSut + "/testverktøy/vedtak-om-overgangsstnad")
28      .then()
29      .statusCode(403)
30      .assertThat()
31      .contentType("text/html");
32
33  // Fault900. A Protected Resource Is Accessible Without Providing Any Authentication.
34  ValidatableResponse res_2 = given().accept("*/")
35      .header("x-EMextraHeader123", "")
36      .contentType("application/json")
37      .body(" { " +
38             "   \"ident\": \"\" " +
39             " } ")
40      .post(baseUrlOfSut + "/testverktøy/vedtak-om-overgangsstnad")
41      .then()
42      .statusCode(200)
43      .assertThat()
44      .contentType("application/json")
45      .body("data", nullValue())
46      .body("status", containsString("FEILET"))
47      .body("melding", containsString("Fant ikke aktiv ident for aktør"))
48      .body("frontendFeilmelding", nullValue())
49      .body("stacktrace", nullValue());
50  String location_vedtak_om_overgangsst_nad_98 = res_2.extract().header("location");
51  assertTrue(isValidURIorEmpty(location_vedtak_om_overgangsst_nad_98));
52 }

```

Fig. 14. Snippet example of generated test for *familie-ba-sak* finding a F900 *Ignore Anonymous* fault. For reasons of space, the calls retrieving the authentication tokens are omitted.

difficult to automatically detect when an API is wrongly returning a 2xx on a failed call. And, even if NLP techniques are used, there is the need to make sure they work for different human languages (e.g., Norwegian Bokmål or Nynorsk, as in this case). Several of our security oracles rely on the correct use of the HTTP protocol (e.g., user errors should return a status code in the 4xx family). When they detect a fault, they could indeed detect a security fault, or, like in this case, they might detect a major issue in the use of HTTP semantics in the tested API. In either case, these are serious faults that need fixing, and that our novel techniques can automatically detect without the need of any natural language processing. Detecting this fault is not a false-positive. It is either a serious security fault, or a serious design/functional fault. In either case, it needs to be fixed. Although security pentesters might not be interested in HTTP compliance-related faults, those would be important faults to fix for quality assurance and software engineers (Masse, 2011).

RQ3: Our novel security oracles were effective at finding new faults in existing real-world APIs. They either detect security vulnerabilities, or serious issues in the use of the HTTP protocol.

6.6. Results for RQ4

In fuzzing, there is a balance to strike between the time spent in generating high coverage test cases (either code coverage and/or REST API schema coverage) and the time spent in crafting ad-hoc test cases

targeted at specific security vulnerabilities. As explained in Section 5, our security-phase is run as a post-processing, after the main fuzzing is completed. In our context, the number of generated tests evaluated for security is fixed, depending on the number of endpoints and the type and quality of the generated tests during the initial fuzzing. If the main fuzzing session is run for 1 h, it is important to evaluate what the additional overhead is introduced by our security phase.

Table 5 shows, for each API in WFD used for RQ3, the overhead in seconds of our additional security phase. To better analyze those results, we also report the 2xx schema coverage (i.e., the percentage of endpoints for which successful 2xx calls were created). If no success call is made, and the execution of test cases ends at the first layer of input validation with a 4xx response, then it would be unlikely that any security testing would be successful, as most of our oracles require the ability to generate success calls to build upon. Fuzzing REST APIs has witnessed significant improvements in the research literature, but it still has many open research problems that need addressing (Zhang and Arcuri, 2023).

As shown in Table 5, in most cases our security phase is fast, taking few tens of seconds, or up to 2 min, which is a reasonable and practical amount of time considering a budget of 1-hour for the initial fuzzing session. On these APIs, the median time is less than 14 s, whereas the average is less than 3 min. Out of 36 APIs in WFD, there are, though, three outliers: *familie-ba-sak* (13 min), *ocvn* (33 min) and *proxyprint* (38 min).

Table 5

For the 36 APIs in WFD, we report the obtained 2xx coverage of their endpoints, and the overhead in seconds of our new security phase.

SUT	#Endpoints	% 2xx Coverage	Overhead
<i>bibliothek</i>	8	20.0	1.2
<i>blogapi</i>	52	30.4	123.7
<i>catwatch</i>	14	49.3	110.2
<i>cwa-verification</i>	5	0.0	0.0
<i>erc20-rest-service</i>	13	7.7	0.4
<i>familie-ba-sak</i>	183	49.5	818.2
<i>features-service</i>	18	21.1	5.4
<i>genome-nexus</i>	23	72.9	37.1
<i>gestaohospital</i>	20	58.3	12.4
<i>http-patch-spring</i>	6	73.3	8.2
<i>languagetool</i>	2	92.9	49.7
<i>market</i>	13	43.1	58.3
<i>microcks</i>	88	46.3	81.9
<i>ocvn</i>	258	84.0	1986.5
<i>ohsome-api</i>	134	2.8	14.8
<i>pay-publicapi</i>	10	57.5	5.1
<i>person-controller</i>	12	43.5	3.0
<i>proxyprint</i>	74	67.7	2325.6
<i>quartz-manager</i>	11	36.4	7.3
<i>reservations-api</i>	7	57.1	91.5
<i>rest-ncs</i>	6	100.0	1.0
<i>rest-news</i>	7	91.4	9.2
<i>rest-scs</i>	11	100.0	1.0
<i>restcountries</i>	22	100.0	6.4
<i>scout-api</i>	49	34.1	21.8
<i>session-service</i>	8	65.3	10.1
<i>spring-actuator-demo</i>	2	100.0	21.7
<i>spring-batch-rest</i>	5	100.0	1.0
<i>spring-ecommerce</i>	27	47.7	156.9
<i>spring-rest-example</i>	9	92.6	35.0
<i>swagger-petstore</i>	19	69.0	2.4
<i>tiltaksgjennomforing</i>	79	8.9	8.7
<i>tracking-system</i>	67	38.7	142.8
<i>user-management</i>	21	66.7	61.2
<i>webgoat</i>	204	16.7	54.3
<i>youtube-mock</i>	1	22.2	0.0
Average		54.6	174.3
Median		53.3	13.6

The cost of security phase is directly dependent on the number of endpoints in the API, and dependent on the number of endpoints for which the fuzzer can create 2xx successful requests. In the case of *familie-ba-sak* and *ocvn*, those are among the largest APIs in WFD (recall Table 2). In particular, *ocvn* is the largest (258 endpoints), where 84% of those endpoints could be covered successfully (see Table 5). As such, for these large APIs it is not unexpected that they require longer time to security test them.

On the other hand, for *proxyprint*, due to its intrinsic characteristics, the average cost of running the HTTP calls (and collecting white-box heuristics from the bytecode instrumentation) is much higher than in other APIs, i.e., each fitness evaluation takes longer. As such, given the same amount of evaluated test cases, the security phase for *proxyprint* takes significantly longer.

Based on these results, from a usability and practical standpoint, in our latest versions of EvoMASTER we provide some further configurable parameters to put timeouts on the security-phase, as a proportion of the search budget (i.e., 10% of the budget, which would be 6 min if the budget is 1 h). If the timeout is reached, the security testing phase is halted (but still keeping the generated tests so far that detect security vulnerabilities), and a warning is issued to the user.

For how long to run the security phase, and what can be considered adequate, strongly depends on the real-world constraints on time and resources in industrial contexts. For example, in many fuzzing contexts (e.g., of parser libraries) it is not uncommon to have fuzzing campaigns of 24 h (e.g., Götz et al., 2024; Huang and Lemieux, 2026). In comparison, security testing phases lasting less than an hour are significantly shorter and manageable.

RQ4: In the large majority of cases, our novel techniques have small, negligible computational overhead.

7. Threats to validity

Our experiments rely on randomized algorithms. To take their randomness into account, experiments were repeated 10 times.

In our empirical study, 52 distinct APIs were used (9 toy examples, 8 examples for security research, and 36 from the WFD corpus). Considering the use of 52 APIs, although this study might be currently the largest study in literature so far on open-source REST APIs, a larger case study would be needed to be able to generalize our results, especially for APIs developed in industry.

Our novel oracles are specific to REST APIs. They do not necessarily generalize to penetration testing tools aimed at web applications. Still, APIs are extremely important, with OWASP creating a specific Top-10 Security Vulnerabilities list for them.²

A state-of-the-art fuzzer such as EvoMASTER is not limited to just security testing. It covers the important aspect of generating executable, high coverage test cases useful for regression testing, as well as detecting functional faults (e.g., crashes and schema mismatches). This is important for the thousands of testers and developers in industry that use EvoMASTER in their development/testing processes. The ability of detecting security vulnerabilities is an extra feature, which is not constrained just for security specialists. As such, when a detected fault turned out to be an HTTP misuse rather than a security vulnerability, it still provides value. Future work might involve the use of LLMs to provide heuristics to classify what kind of fault was detected (e.g., Ahmad et al., 2026a; Ahmad and Zhang, 2025; Ahmad et al., 2026b).

Our novel oracles are not specific nor tailored to any fuzzer. In our experiments, we used EvoMASTER, but in theory any other state-of-the-art fuzzer could have been used. How easy or difficult it would be for our novel techniques to be integrated in other fuzzers is a matter that would require further investigation.

8. Conclusion

In this paper, we have presented seven novel oracles aimed at detecting authorization-related faults, as well as integrating two common injection attacks, in REST APIs. These techniques were used to detect hundreds of new faults in an empirical study consisting of 52 REST APIs.

Our novel techniques were implemented as an extension of EvoMASTER, a state-of-the-art open-source fuzzer. Once our novel techniques detect new faults, in contrast to other techniques/tools, executable test cases (e.g., in JUnit format) are automatically generated to help practitioners in industry to reproduce the faults and debug them. These generated tests can hence also be used for regression testing.

Future work will aim at designing new automated oracles to identify other kinds of security faults in REST APIs, and carry out empirical studies in industry (for example at Fortune 500 enterprises that use EvoMASTER, like Volkswagen (Arcuri et al., 2025a) and Meituan (Zhang et al., 2025)).

Our extension to EvoMASTER is released as open-source at www.evomaster.org. The novel techniques presented in this paper are now activated by default in EvoMASTER's latest releases (since version 5.2.0), which have already been downloaded hundreds of times.

CRedit authorship contribution statement

Omur Sahin: Writing – review & editing, Validation, Software, Methodology, Conceptualization. **Man Zhang:** Writing – review & editing, Software, Conceptualization. **Andrea Arcuri:** Writing – original draft, Validation, Software, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Andrea Arcuri reports financial support was provided by European Research Council. Omur Sahin reports financial support was provided by Scientific and Technological Research Council of Turkey. Man Zhang reports financial support was provided by National Natural Science Foundation of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is funded by the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (EAST project, grant agreement No. 864972). Omur Sahin is supported by the TÜBİTAK 2219 International Postdoctoral Research Fellowship Program (Project ID: 1059B192300060). Man Zhang is supported by the National Science Foundation of China (grant agreement No. 62502022).

Data availability

All the techniques presented in this paper are implemented as part of EvoMASTER. The fuzzer EvoMASTER is open-source on GitHub, (<https://github.com/WebFuzzing/EvoMaster>) where each new release is automatically published on Zenodo for long-term storage (e.g., Arcuri et al. (2026b)).

WFD is open-source on GitHub, (<https://github.com/WebFuzzing/Dataset>) with as well each new release automatically uploaded to Zenodo for long-term storage (e.g., Arcuri et al. (2026a)).

The vulnerable APIs used for RQ2 are all open-source on GitHub, with IDs as specified in Table 1.

References

- Addula, S.R., Norozpour, S., Amin, M., 2025. Risk assessment for identifying threats, vulnerabilities and countermeasures in cloud computing. *Jordanian J. Inform. Comput.* 2025 (1), 37–48.
- Ahmad, N., Sehar, U., Zhou, J., Peng, Y., Zhang, C., 2026a. Harnessing LLMs for cyber-resilience a systematic survey in threat mitigation and future trajectories. *IEEE Access*.
- Ahmad, N., Zhang, C., 2025. Interpretable vulnerability detection in LLMs: A BERT-based approach with SHAP explanations. *Comput. Mater. Contin.* 85 (2), 3321.
- Ahmad, N., Zhang, C., Sehar, U., 2026b. Mitigating adversarial obfuscation in named entity recognition with robust SecureBERT finetuning. *Comput. Mater. Contin.* 87 (1).
- Appelt, D., Nguyen, C.D., Briand, L.C., Alshahwan, N., 2014. Automated testing for SQL injection vulnerabilities: an input mutation approach. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. pp. 259–269.
- Arcuri, A., 2017. Restful API automated test case generation. In: *IEEE International Conference on Software Quality, Reliability and Security*. QRS, IEEE, pp. 9–20.
- Arcuri, A., 2019. Restful API automated test case generation with EvoMaster. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 28 (1), 3.
- Arcuri, A., Galeotti, J.P., 2020. Handling SQL databases in automated system test generation. *TOSEM* 29 (4), 1–31.
- Arcuri, A., Poth, A., Rjolli, O., 2025a. Introducing black-box fuzz testing for REST APIs in industry: Challenges and solutions. In: *IEEE International Conference on Software Testing, Verification and Validation*. ICST.
- Arcuri, A., Sahin, O., Zhang, M., 2025b. Fuzzing for detecting access policy violations in REST APIs. In: *IEEE International Symposium on Software Reliability Engineering*. ISSRE.
- Arcuri, A., Zhang, M., Belhadi, A., Marculescu, B., Golmohammadi, A., Galeotti, J.P., Seran, S., 2023a. Building an open-source system test generation tool: lessons learned and empirical analyses with EvoMaster. *Softw. Qual. J.* 1–44.
- Arcuri, A., Zhang, M., Golmohammadi, A., Belhadi, A., Galeotti, J.P., Marculescu, B., Seran, S., 2023b. EMB: A curated corpus of web/enterprise applications and library support for software testing research. In: *2023 IEEE Conference on Software Testing, Verification and Validation*. ICST, IEEE, pp. 433–442.
- Arcuri, A., Zhang, M., Şahin, Ö., Golmohammadi, A., Belhadi, A., Seran, S., Duman, O., Galeotti, J.P., Assres, G.M., Ghianni, H., 2026a. WebFuzzing/Dataset: v4.1.0. <http://dx.doi.org/10.5281/zenodo.18196038>.
- Arcuri, A., Zhang, M., Seran, S., Belhadi, A., Galeotti, J.P., Bogdan, Golmohammadi, A., Duman, O., Philip, Aldasoro, A., Şahin, Ö., Shalmani, M.T., Castagna, F., López, A.M., Ghianni, H., aszyrej, Rodriguez, M., Castagna, F.N., Roca, L.M., IvaK, Panichella, A., Niemeyer, K., Maugeri, M., 2026b. WebFuzzing/EvoMaster: v5.0.2. <http://dx.doi.org/10.5281/zenodo.18262322>.
- Arcuri, A., Zhang, M., Seran, S., Galeotti, J.P., Golmohammadi, A., Duman, O., Aldasoro, A., Ghianni, H., 2025c. Tool report: EvoMaster—black and white box search-based fuzzing for REST, GraphQL and RPC APIs. *Autom. Softw. Eng.* 32 (1), 1–11.
- Atlidakis, V., Godefroid, P., Polishchuk, M., 2019. RESTler: Stateful REST API fuzzing. In: *ACM/IEEE International Conference on Software Engineering*. ICSE, pp. 748–758.
- Atlidakis, V., Godefroid, P., Polishchuk, M., 2020. Checking security properties of cloud service rest apis. In: *IEEE International Conference on Software Testing, Verification and Validation*. ICST, IEEE, pp. 387–397.
- Aziz, B., Bader, M., Hippolyte, C., 2016. Search-based sql injection attacks testing using genetic programming. In: *Genetic Programming: 19th European Conference, EuroGP 2016, Porto, Portugal, March 30–April 1, 2016, Proceedings 19*. Springer, pp. 183–198.
- Barr, E.T., Harman, M., McMinn, P., Shahbaz, M., Yoo, S., 2015. The oracle problem in software testing: A survey. *TSE* 41 (5), 507–525.
- Bogner, J., Kotstein, S., Abajirov, D., Ernst, T., Merkel, M., 2024. RESTruler: towards automatically identifying violations of restful design rules in web APIs. In: *2024 IEEE 21st International Conference on Software Architecture*. ICASA, IEEE, pp. 123–134.

- Decrop, A., Papadakis, M., Perrouin, G., 2026. Analyzing status code misuses in REST API specifications. In: Proceedings of the 26th International Conference on Web Engineering, ICWE 2026.
- Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., Rass, S., 2024. {PentestGPT}: Evaluating and harnessing large language models for automated penetration testing. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 847–864.
- Deng, G., Zhang, Z., Li, Y., Liu, Y., Zhang, T., Liu, Y., Yu, G., Wang, D., 2023. {NAUTILUS}: Automated {RESTful} {API} vulnerability detection. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 5593–5609.
- Di Meglio, S., Pontillo, V., Starace, L.L.L., 2025. REST in pieces: Restful design rule violations in student-built web apps. In: Euromicro Conference on Software Engineering and Advanced Applications. Springer, pp. 191–200.
- Du, W., Li, J., Wang, Y., Chen, L., Zhao, R., Zhu, J., Han, Z., Wang, Y., Xue, Z., 2024. Vulnerability-oriented testing for restful apis. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, pp. 739–755.
- Foley, M., Maffei, S., 2025. APRL: Deep reinforcement learning for REST API fuzzing. In: Thirty-Ninth Conference on Artificial Intelligence (AAAI 2025).
- Golmohammadi, A., Zhang, M., Arcuri, A., 2023. Testing RESTful APIs: A survey. ACM Trans. Softw. Eng. Methodol. <http://dx.doi.org/10.1145/3617175>.
- Görz, P., Schilling, J., Wu, X., Chen, H., Gopinath, R., 2024. SBFT tool competition 2024—Fuzzing track. In: Proceedings of the 17th ACM/IEEE International Workshop on Search-Based and Fuzz Testing. pp. 25–28.
- Hatfield-Dodds, Z., Dygalo, D., 2022. Deriving semantics-aware fuzzers from web API schemas. In: 2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). IEEE, pp. 345–346.
- Huang, M., Lemieux, C., 2026. Directed or undirected: Investigating fuzzing strategies in a CI/CD setup. ACM Trans. Softw. Eng. Methodol.
- Hydara, I., Sultan, A.B.M., Zulzalil, H., Admodisastro, N., 2015. Current state of research on cross-site scripting (XSS)—a systematic literature review. Inf. Softw. Technol. 58, 170–186.
- Kaur, B., 2026. Broken object level authorization in the wild: An empirical taxonomy from 100+ bug bounty disclosures. arXiv preprint [arXiv:2605.25865](https://arxiv.org/abs/2605.25865).
- Kieyzen, A., Guo, P., Jayaraman, K., Ernst, M., 2009. Automatic creation of SQL injection and cross-site scripting attacks. In: ACM/IEEE International Conference on Software Engineering. ICSE, pp. 199–209.
- Kim, M., Sinha, S., Orso, A., 2023. Adaptive rest api testing with reinforcement learning. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 446–458.
- Kim, M., Sinha, S., Orso, A., 2025a. LlamaRestTest: Effective REST API testing with small language models. In: ACM Symposium on the Foundations of Software Engineering. FSE.
- Kim, M., Stennett, T., Sinha, S., Orso, A., 2025b. A multi-agent approach for REST API testing with semantic graphs and LLM-driven inputs. ICSE.
- Kim, M., Xin, Q., Sinha, S., Orso, A., 2022. Automated test generation for REST APIs: No time to rest yet. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. In: ISSTA 2022, Association for Computing Machinery, New York, NY, USA, pp. 289–301. <http://dx.doi.org/10.1145/3533767.3534401>.
- Laranjeiro, N., Agnello, J., Bernardino, J., 2021. A black box tool for robustness testing of REST services. IEEE Access 9, 24738–24754.
- Liu, M., Li, K., Chen, T., 2020. DeepSQLI: Deep semantic learning for testing SQL injection. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 286–297.
- Liu, Y., Li, Y., Deng, G., Liu, Y., Wan, R., Wu, R., Ji, D., Xu, S., Bao, M., 2022. Morest: Model-based restful API testing with execution feedback. In: ACM/IEEE International Conference on Software Engineering. ICSE.
- Lyu, C., Xu, J., Ji, S., Zhang, X., Wang, Q., Zhao, B., Pan, G., Cao, W., Chen, P., Beyah, R., 2023. MINER: A hybrid data-driven approach for REST API fuzzing. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 4517–4534.
- Marculescu, B., Zhang, M., Arcuri, A., 2022. On the faults found in REST APIs by automated test generation. ACM Trans. Softw. Eng. Methodol. (TOSEM) 31 (3), 1–43.
- Martin, E., Xie, T., Yu, T., 2006. Defining and measuring policy coverage in testing access control policies. In: Information and Communications Security: 8th International Conference, ICICS 2006, Raleigh, NC, USA, December 4–7, 2006. Proceedings 8. Springer, pp. 139–158.
- Martin-Lopez, A., Segura, S., Ruiz-Cortés, A., 2019. Test coverage criteria for restful web APIs. In: Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation. pp. 15–21.
- Martin-Lopez, A., Segura, S., Ruiz-Cortés, A., 2021. RESTest: Automated Black-Box Testing of RESTful Web APIs. In: ACM Int. Symposium on Software Testing and Analysis. ISSTA, ACM, pp. 682–685.
- Masse, M., 2011. REST API design rulebook: designing consistent RESTful web service interfaces. O'Reilly Media, Inc..
- Pasca, E.M., Delinschi, D., Erdei, R., Matei, O., 2025. LLM-driven, self-improving framework for security test automation: Leveraging karate DSL for augmented API resilience. IEEE Access.
- Poth, A., Rrjolli, O., Arcuri, A., 2025. Technology adoption performance evaluation applied to testing industrial REST APIs. Autom. Softw. Eng. 32 (1), 5.
- Rooijackers, T., Nijsten, A., Daniele, C., Weitenberg, E., Groenewegen, R., Melissen, A., 2025. WuppieFuzz: Coverage-guided, stateful REST API fuzzing. arXiv preprint [arXiv:2512.15554](https://arxiv.org/abs/2512.15554).
- Sahin, O., Zhang, M., Arcuri, A., 2025. WFC/WFD: web fuzzing commons, dataset and guidelines to support experimentation in REST API fuzzing. [arXiv:2509.01612](https://arxiv.org/abs/2509.01612). URL <https://arxiv.org/abs/2509.01612>.
- Sahin, O., Zhang, M., Arcuri, A., 2026. Enhancing REST API fuzzing with access policy violation checks and injection attacks. arXiv preprint [arXiv:2604.00702](https://arxiv.org/abs/2604.00702).
- Santos Filho, A., Rodríguez, R.J., Feitosa, E.L., 2025. Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored petri nets transformation. Int. J. Inf. Secur. 24 (2), 83.
- Sartaj, H., Ali, S., Gjøby, J.M., 2024. REST API testing in DevOps: A study on an evolving healthcare IoT application. arXiv:2410.12547. URL <https://arxiv.org/abs/2410.12547>.
- Seran, S., Bhandari, G.P., Arcuri, A., 2026. Detecting Server-Side Request Forgery (SSRF) Vulnerabilities In REST API Fuzz Testing. In: 19th Search-Based and Fuzz Testing Workshop (SBFT '26).
- Shen, X., Wang, L., Li, Z., Chen, Y., Zhao, W., Sun, D., Wang, J., Ruan, W., 2025. Pentestagent: Incorporating llm agents to automated penetration testing. In: Proceedings of the 20th ACM Asia Conference on Computer and Communications Security. pp. 375–391.
- Vigliani, E., Dallago, M., Ceccato, M., 2020. RESTTESTGEN: Automated black-box testing of RESTful APIs. In: IEEE International Conference on Software Testing, Verification and Validation. ICST, IEEE.
- Wang, Y., Xu, Y., 2024. Beyond REST: Introducing APIF for comprehensive API vulnerability fuzzing. In: Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses. pp. 435–449.
- Wu, H., Xu, L., Niu, X., Nie, C., 2022. Combinatorial testing of restful APIs. In: ACM/IEEE International Conference on Software Engineering. ICSE.
- Zhang, M., Arcuri, A., 2023. Open problems in fuzzing RESTful APIs: A comparison of tools. TOSEM <http://dx.doi.org/10.1145/3597205>.
- Zhang, M., Arcuri, A., Li, Y., Liu, Y., Xue, K., 2023. White-box fuzzing RPC-based APIs with EvoMaster: An industrial case study. ACM Trans. Softw. Eng. Methodol. 32 (5), 1–38.
- Zhang, M., Arcuri, A., Li, Y., Liu, Y., Xue, K., Wang, Z., Huo, J., Huang, W., 2025. Fuzzing microservices: A series of user studies in industry on industrial systems with evomaster. Sci. Comput. Program. 103322.
- Zhang, M., Arcuri, A., Teng, P., Xue, K., Wang, W., 2024. Seeding and mocking in white-box fuzzing enterprise RPC apis: An industrial case study. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. pp. 2024–2034.
- Zhang, M., Shen, C., Arcuri, A., Yue, T., 2026. Detecting and mitigating flakiness in REST API fuzzing. arXiv:2603.28452. URL <https://arxiv.org/abs/2603.28452>.